

Java 25 — PEM Encodings of Cryptographic Objects (Preview)

Java Version: 25

Year: 2025

Feature: PEM Encodings of Cryptographic Objects (Preview)

UniqueClassName: PEMEncodings

1. Definition

PEM Encodings of Cryptographic Objects is a **preview feature introduced in Java 25** that provides a standard API for encoding and decoding cryptographic objects using **PEM (Privacy-Enhanced Mail)** format.

PEM is a text-based representation commonly used for cryptographic material such as:

- Public keys
- Private keys
- Certificates
- Certificate signing requests (CSR)
- Other supported cryptographic objects

Instead of manually converting DER/ASN.1 binary data to Base64 and adding PEM headers and footers, the Java API provides a standardized way to perform PEM encoding and decoding.

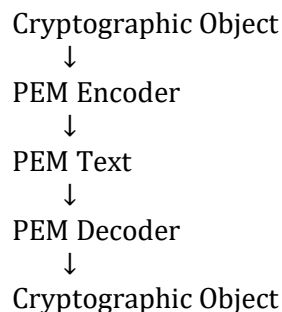
A typical PEM object looks like:

```
-----BEGIN PUBLIC KEY-----
MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8A...
-----END PUBLIC KEY-----
```

2. Why this feature is useful

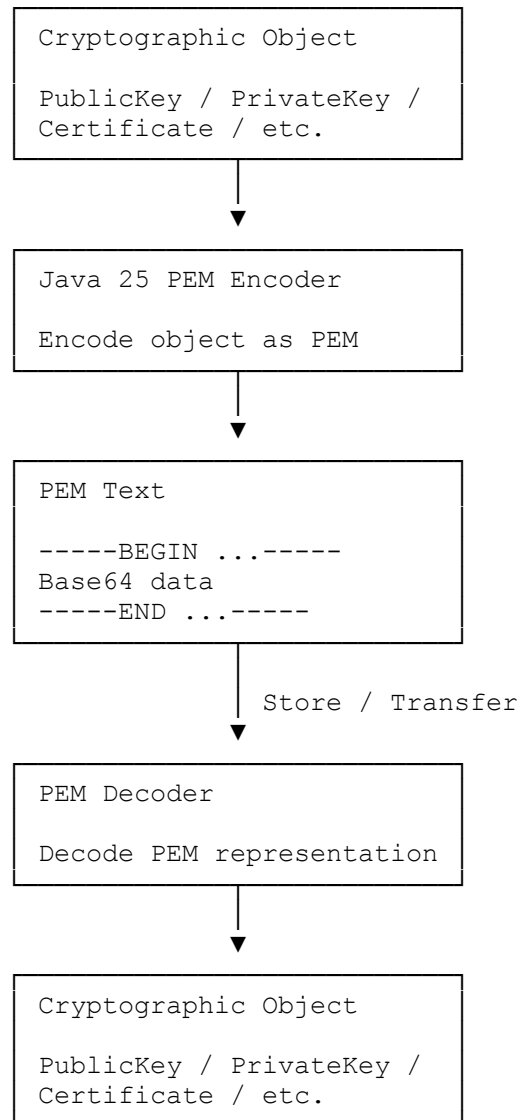
Before this API, Java applications commonly needed to implement PEM handling themselves or rely on third-party libraries.

The Java 25 API makes the workflow conceptually:



This is particularly useful when Java applications exchange keys and certificates with systems such as OpenSSL, TLS infrastructure, Kubernetes, cloud platforms, and other security tools.

3. Flowchart



4. Java Code Example

Because this is a **preview API**, the exact API usage should be compiled and run with Java's preview-feature options.

A representative PEM encoding/decoding example is:

```

import java.security.KeyPair;
import java.security.KeyPairGenerator;
import java.security.PrivateKey;
import java.security.PublicKey;

import java.security.PEMDecoder;
import java.security.PEMEncoder;

public class PEMEncodings {

    public static void main(String[] args) throws Exception {

        // 1. Generate an RSA key pair
        KeyPairGenerator generator =
            KeyPairGenerator.getInstance("RSA");

        generator.initialize(2048);

        KeyPair keyPair = generator.generateKeyPair();

        PublicKey publicKey = keyPair.getPublic();
        PrivateKey privateKey = keyPair.getPrivate();

        // 2. Encode public key as PEM
        String publicPem = PEMEncoder.of()
            .encodeToString(publicKey);

        System.out.println("PUBLIC KEY:");
        System.out.println(publicPem);

        // 3. Encode private key as PEM
        String privatePem = PEMEncoder.of()
            .encodeToString(privateKey);

        System.out.println("PRIVATE KEY:");
        System.out.println(privatePem);

        // 4. Decode public key
        PublicKey decodedPublicKey =
            PEMDecoder.of()
                .decode(publicPem, PublicKey.class);

        // 5. Decode private key
        PrivateKey decodedPrivateKey =
            PEMDecoder.of()
                .decode(privatePem, PrivateKey.class);

        // 6. Verify the decoded objects
        System.out.println(
            "Public key algorithm: "
                + decodedPublicKey.getAlgorithm());

        System.out.println(
            "Private key algorithm: "
                + decodedPrivateKey.getAlgorithm());
    }
}

```

Expected PEM output

The public key will have a structure similar to:

```
-----BEGIN PUBLIC KEY-----
MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8A...
...
-----END PUBLIC KEY-----
```

The private key will have a structure similar to:

```
-----BEGIN PRIVATE KEY-----
MIIEvQIBADANBgkqhkiG9w0BAQEFAASC BK...
...
-----END PRIVATE KEY-----
```

The Base64 contents will differ each time because a new RSA key pair is generated.

5. Simplified API Concept

The important classes are:

```
PEMEncoder
├── encode(...)
└── encodeToString(...)

PEMDecoder
└── decode(...)
```

Conceptually:

```
String pem = PEMEncoder.of()
    .encodeToString(publicKey);
```

converts:

```
PublicKey
↓
DER representation
↓
Base64
↓
PEM text
```

And:

```
PublicKey key = PEMDecoder.of()  
    .decode(pem, PublicKey.class);
```

performs the reverse operation:

```
PEM text  
↓  
Base64 decoding  
↓  
DER representation  
↓  
PublicKey
```

6. Key takeaway

Java 25's PEM Encodings preview feature standardizes PEM processing in the JDK, reducing the need for application-specific PEM parsing and Base64/header/footer handling.

Note: Since this is a **preview API**, preview features are subject to change and require the appropriate Java 25 preview compilation/runtime options.

Java 25 — Stable Values (Preview)

Java Version: 25

Year: 2025

Feature: Stable Values (Preview)

UniqueClassName: StableValues

1. Definition

Stable Values is a **preview feature introduced in Java 25** that provides a new way to represent values that are:

- initialized **at most once**,
- remain **unchanged after initialization**, and
- can be initialized **lazily**.

The feature introduces the `StableValue<T>` abstraction in the `java.lang` package.

It is especially useful when an object needs to be created only when it is first required, while guaranteeing that the same value is subsequently reused.

Traditional approach

Without Stable Values, lazy initialization might look like:

```
private volatile Service service;

Service getService() {
    if (service == null) {
        service = createService();
    }
    return service;
}
```

This can require synchronization or other concurrency mechanisms to make initialization safe.

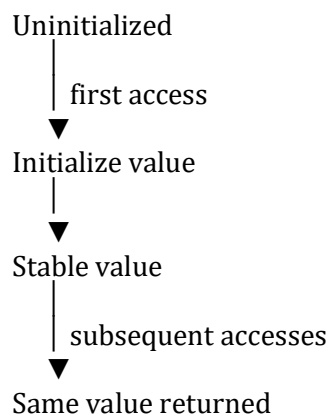
With Stable Values

The intention becomes much simpler:

```
StableValue<Service> service = StableValue.of();

Service getService() {
    return service.orElseSet(this::createService);
}
```

Conceptually:



2. Main Characteristics

① Lazy initialization

The value does not have to be created when the containing object is constructed.

StableValue.of()

starts in an uninitialized state.

② Initialized only once

Once the value is established, it cannot be replaced with another value.

null/uninitialized



Value A



Value A forever

③ Thread-safe initialization

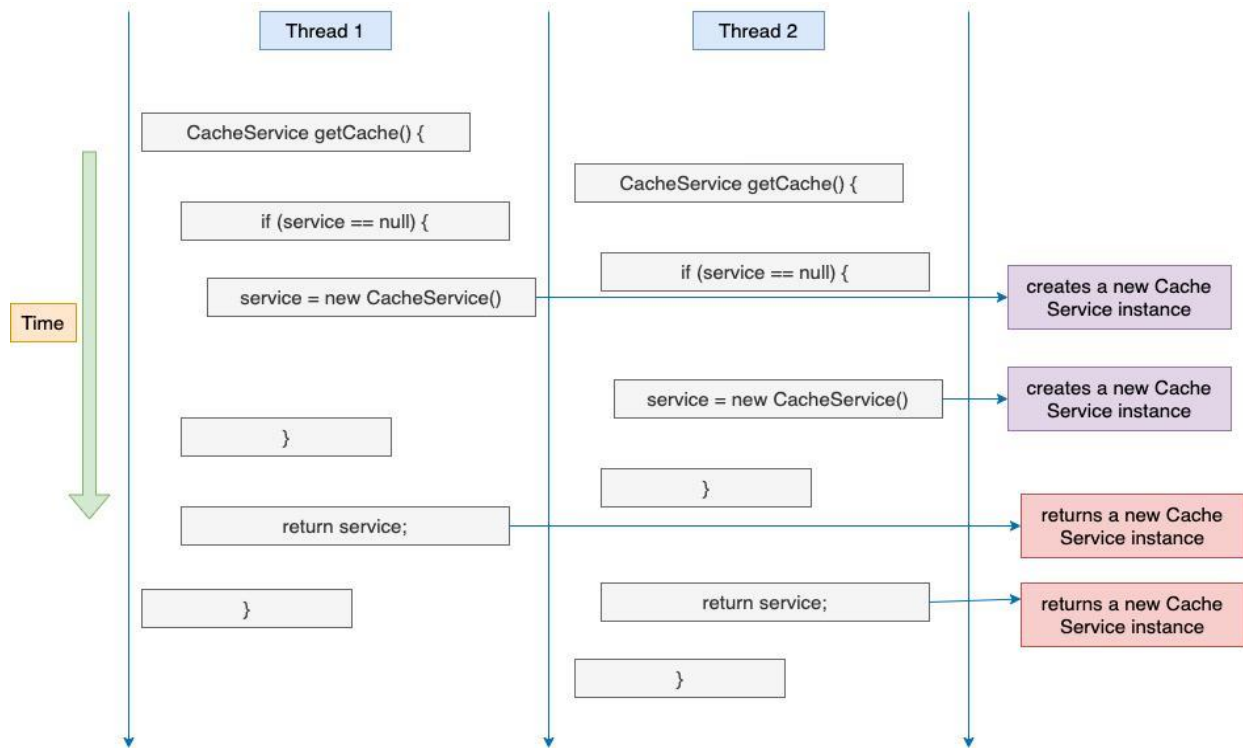
Stable Values are designed to support safe lazy initialization in concurrent applications without requiring developers to implement their own synchronization logic.

④ Useful for immutable configuration

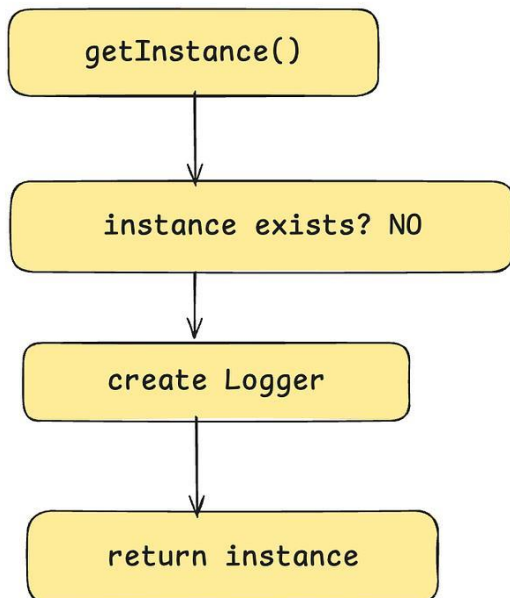
They are particularly useful for values such as:

- expensive service objects,
 - configuration objects,
 - lookup tables,
 - compiled patterns,
 - immutable caches,
 - application-wide constants that are expensive to construct.
-

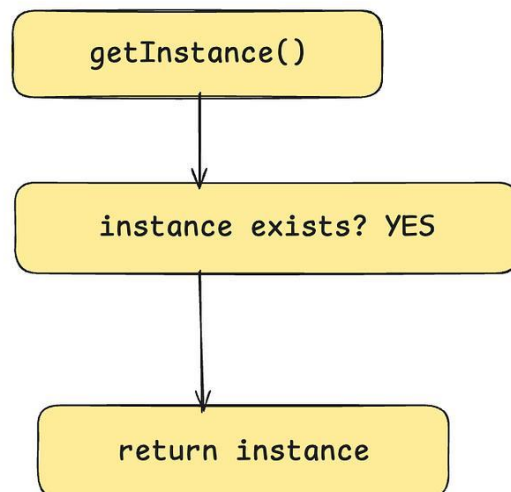
3. Flowchart

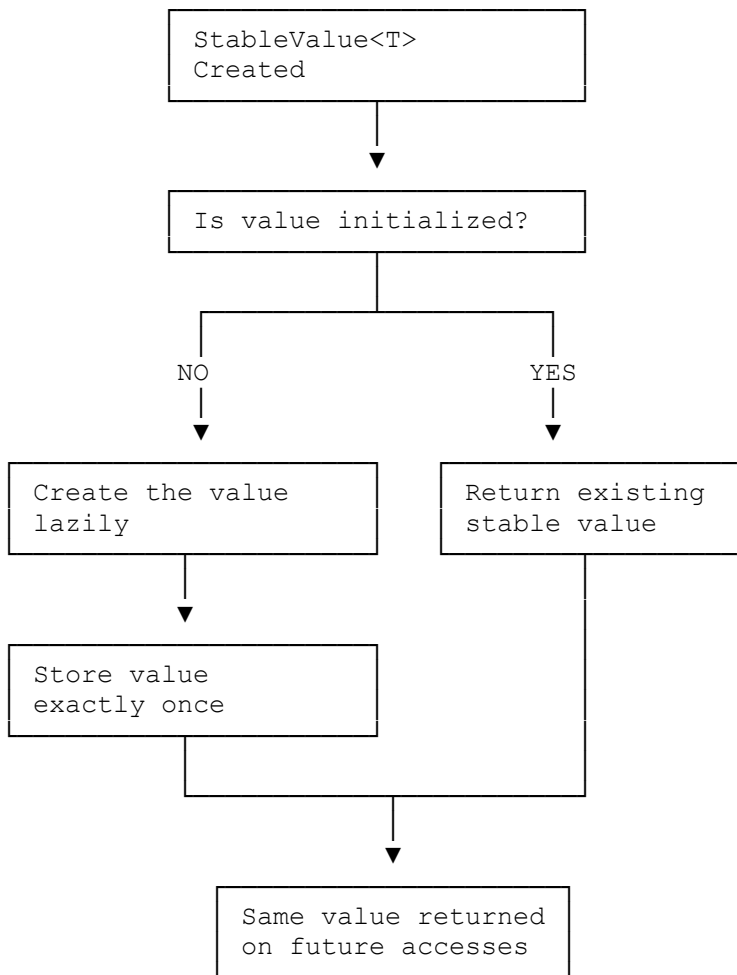


First Call:



Second Call:





4. Java Code Example

```
import java.lang.StableValue;

public class StableValues {

    private static final StableValue<String> CONFIG =
        StableValue.of();

    public static String getConfig() {

        return CONFIG.orElseSet(() -> {
            System.out.println("Initializing configuration...");
            return "Production Configuration";
        });
    }
}
```

```

public static void main(String[] args) {

    System.out.println("Application started");

    // First access
    String config1 = getConfig();

    // Second access
    String config2 = getConfig();

    System.out.println("Config 1: " + config1);
    System.out.println("Config 2: " + config2);

    System.out.println(
        "Same object: " + (config1 == config2)
    );
}
}

```

Conceptual output

```

Application started
Initializing configuration...
Config 1           : Production Configuration
Config 2           : Production Configuration
Same object        : true

```

Notice that:

```

Initializing configuration...

```

is printed **only once**.

The second call:

```

String config2 = getConfig();

```

reuses the already initialized stable value.

5. Expensive Object Example

Stable Values become more interesting when the initialization is expensive.

```

import java.lang.StableValue;

public class StableValues {

    private static final StableValue<DatabaseConnection> CONNECTION =
        StableValue.of();
}

```

```

static DatabaseConnection getConnection() {

    return CONNECTION.orElseSet(() -> {

        System.out.println("Creating database connection...");

        return new DatabaseConnection(
            "jdbc:postgresql://localhost:5432/mydb"
        );
    });
}

public static void main(String[] args) {

    DatabaseConnection connection1 =
        getConnection();

    DatabaseConnection connection2 =
        getConnection();

    System.out.println(
        connection1 == connection2
    );
}

static class DatabaseConnection {

    private final String url;

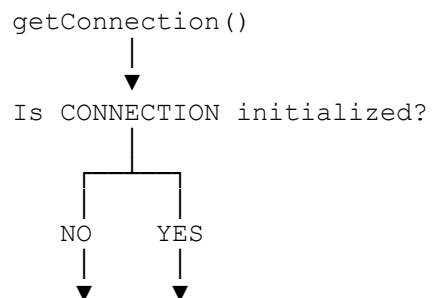
    DatabaseConnection(String url) {
        this.url = url;
    }

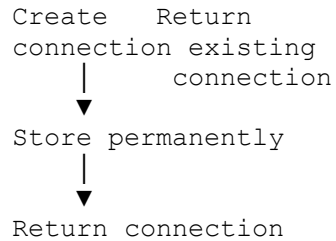
    @Override
    public String toString() {
        return "DatabaseConnection{" +
            "url='" + url + '\'' +
            '}';
    }

}
}

```

Conceptually:





6. Stable Value vs final

A normal final field must normally be initialized during construction:

```
public class Example {  
    private final String value;  
  
    public Example() {  
        value = loadValue();  
    }  
}
```

This means the value is initialized eagerly.

With a stable value:

```
private final StableValue<String> value =  
    StableValue.of();
```

the actual String can be initialized later:

```
String getValue() {  
    return value.orElseSet(this::loadValue);  
}
```

Comparison

Feature	final field	StableValue
Can change after initialization?	No	No
Lazy initialization	No, not by itself	Yes
Initialized during construction	Usually	Not necessarily
Designed for safe lazy	No	Yes

initialization		
Preview in Java 25	No	Yes

7. Important API Concept

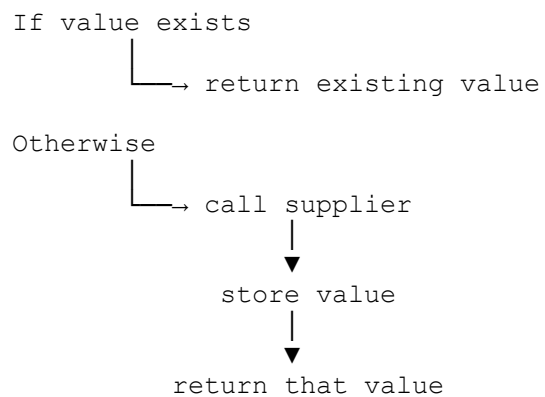
The basic lifecycle is:

```
StableValue<T> value = StableValue.of();
```

Then:

```
value.orElseSet(supplier);
```

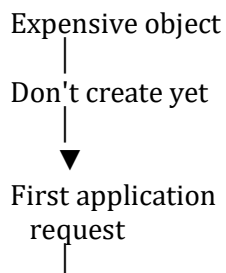
means approximately:

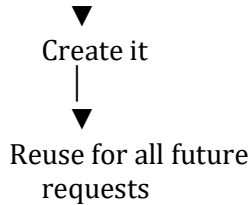


The crucial property is that after initialization, the stable value **does not change**.

8. Why Java Introduced Stable Values

Stable Values address a common pattern in Java applications:





Examples include:

```
StableValue<Configuration>
StableValue<Logger>
StableValue<DatabaseConnection>
StableValue<Pattern>
StableValue<Service>
StableValue<LookupTable>
```

This makes **lazy + immutable + safely published state** easier to express.

9. One-Line Summary

Stable Values (Java 25 Preview) provide a standard JDK mechanism for **lazily initializing a value exactly once and then safely reusing that unchanged value**, simplifying a common form of lazy initialization in concurrent Java applications.

Java 25 — Remove 32-bit x86 Port

Java Version: 25

Year: 2025

Feature: Remove 32-bit x86 Port

UniqueClassName: RemoveX86Port

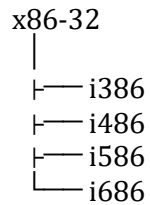
1. Definition

Remove 32-bit x86 Port is a **Java 25 platform change** that removes the HotSpot/JDK support for running Java on **32-bit x86 (x86-32 / i686) systems**.

This follows the earlier deprecation of the 32-bit x86 port in **JDK 24**. With Java 25, the 32-bit x86 port is removed rather than merely deprecated.

What is 32-bit x86?

32-bit x86 refers to the traditional Intel/AMD architecture using **32-bit addresses and registers**, commonly associated with processors and operating systems such as:



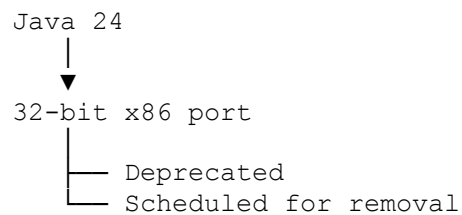
Modern Java deployments overwhelmingly use **64-bit architectures**, such as:

x86-64 / AMD64
AArch64 / ARM64

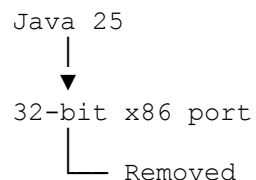
Therefore, Java 25 removes the obsolete 32-bit x86 port to reduce maintenance and platform-specific complexity.

2. What Changed?

Before Java 25



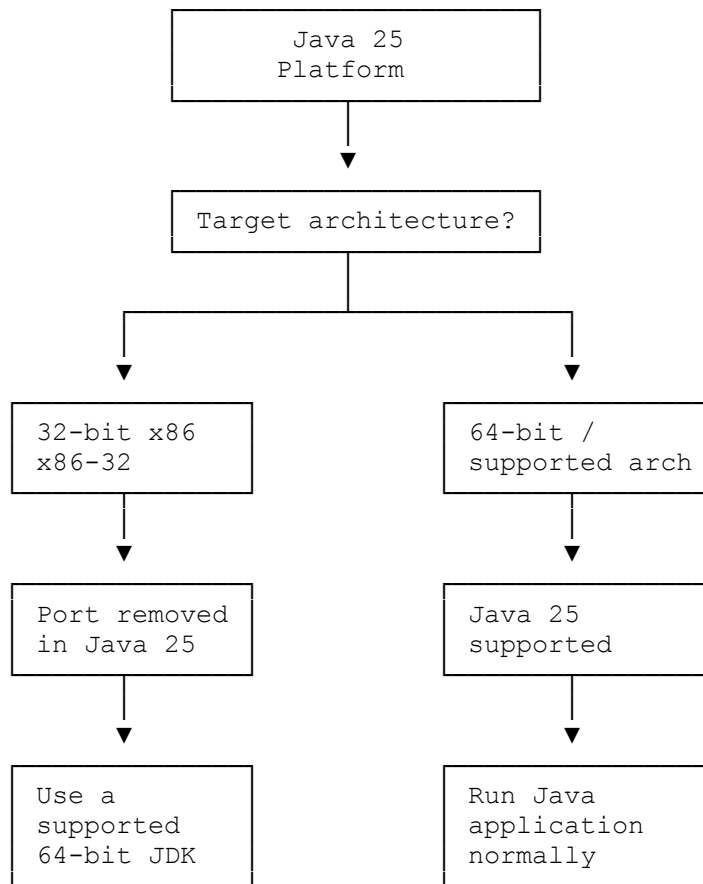
Java 25



The important distinction is:

Java source code itself is not being removed. The 32-bit x86 runtime/platform port is removed.

3. Flowchart



4. Java Code Example

There is **no new Java language syntax or Java API** associated with this feature.

That is important.

You do **not** write something such as:

```
RemoveX86Port.remove();
```

Instead, the change affects the **JDK runtime/platform**, not application source code.

For example, an ordinary Java 25 program remains completely normal:

```
public class RemoveX86Port {
```

```

public static void main(String[] args) {

    System.out.println("Java application running");

    long value = 10_000_000_000L;

    System.out.println("Value = " + value);
}
}

```

On a supported 64-bit Java 25 environment, the program can execute normally.

5. Demonstrating the Architecture

Although the feature does not introduce a Java API, you can inspect the architecture on which the JVM is running:

```

public class RemoveX86Port {

    public static void main(String[] args) {

        System.out.println(
            "OS Architecture: "
            + System.getProperty("os.arch")
        );

        System.out.println(
            "OS: "
            + System.getProperty("os.name")
        );

        System.out.println(
            "Java Version: "
            + System.getProperty("java.version")
        );
    }
}

```

A modern 64-bit system might produce output similar to:

```

OS Architecture   : amd64
OS                : Windows 11
Java Version      : 25

```

Or on an ARM64 machine:

```

OS Architecture: aarch64
OS              : Linux

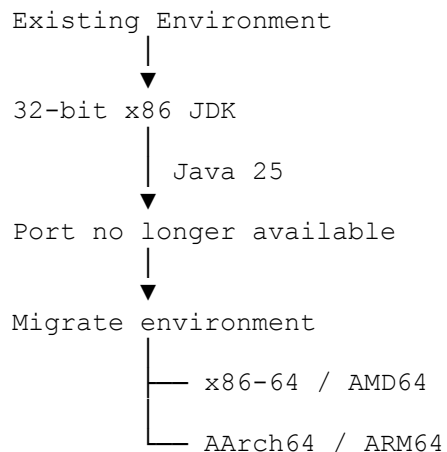
```

Java Version : 25

The exact os.arch value depends on the JDK and operating system.

6. What Developers Need to Do

If an application is currently deployed on a **32-bit x86 operating system/JDK**, Java 25 requires a migration to a supported platform.



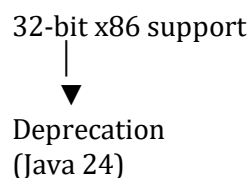
The application source code generally does **not** need to be rewritten merely because the 32-bit x86 port was removed. The main issue is the **runtime/platform architecture** and any native dependencies.

7. Why Was It Removed?

Removing an obsolete platform port provides several benefits:

- **Less JDK maintenance**
- **Reduced HotSpot platform-specific code**
- **Simpler testing and CI**
- **Lower implementation complexity**
- **Greater focus on modern 64-bit architectures**

The transition was deliberate:



↓
Removal
(Java 25)

8. Java 24 vs Java 25

Java Version	Status of 32-bit x86
Java 23	Supported/deprecated transition underway
Java 24	Deprecated for removal
Java 25	Removed

9. Key Takeaway

Java 25 removes the JDK/HotSpot 32-bit x86 port. It is a platform-level change, not a new Java language feature or API. Applications should run on supported 64-bit architectures such as x86-64 or AArch64.

Java 25 — Structured Concurrency (Fifth Preview)

Java Version: 25

Year: 2025

Feature: Structured Concurrency (Fifth Preview)

UniqueClassName: StructuredConcurrency

1. Definition

Structured Concurrency in Java 25 is a **fifth preview** of the Structured Concurrency API, introduced through **JEP 505**.

It treats a group of related concurrent tasks as a single unit of work. Instead of starting independent threads and managing their lifecycle separately, you place related subtasks inside a **structured scope**.

The key idea is:

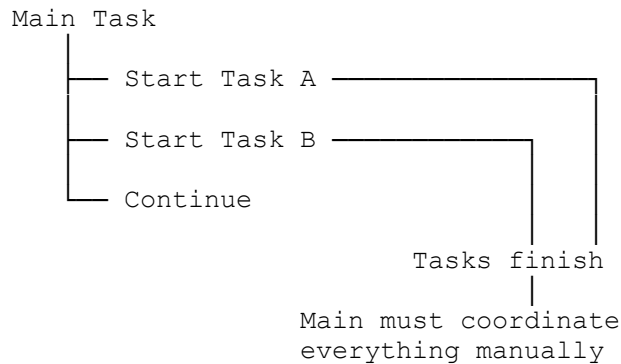
The lifetime of concurrent subtasks should be contained within the lexical scope of the operation that created them.

This makes concurrent code easier to:

- understand,
 - cancel,
 - handle failures,
 - propagate results,
 - and reason about task lifetimes.
-

2. Problem Without Structured Concurrency

Traditional concurrent code can look like this:



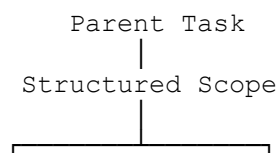
The parent task can accidentally finish while child tasks are still running.

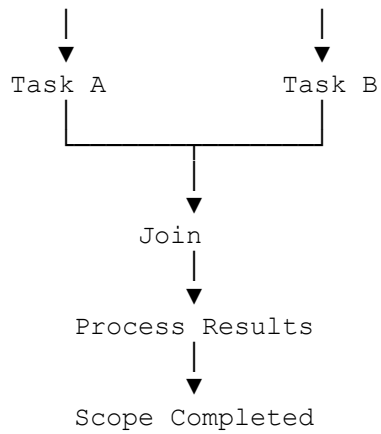
This can lead to problems such as:

- leaked tasks,
 - difficult cancellation,
 - unclear exception handling,
 - complicated coordination.
-

3. Structured Concurrency Approach

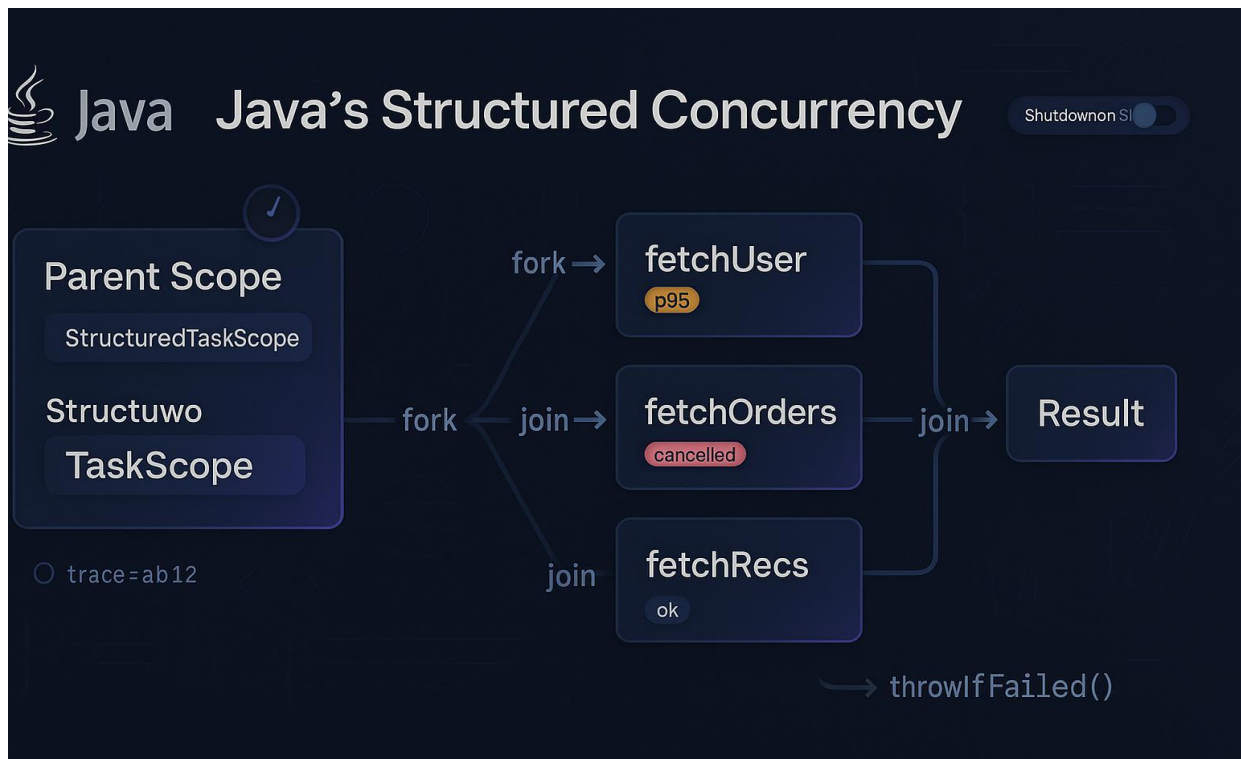
With Structured Concurrency:

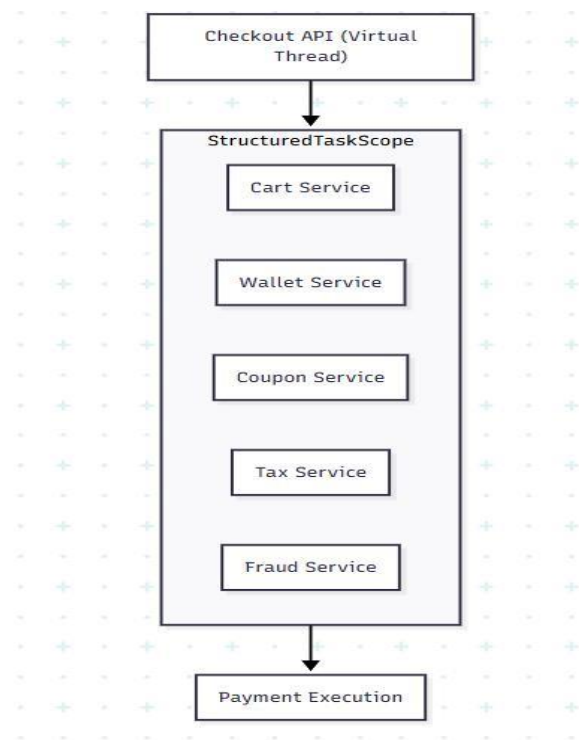
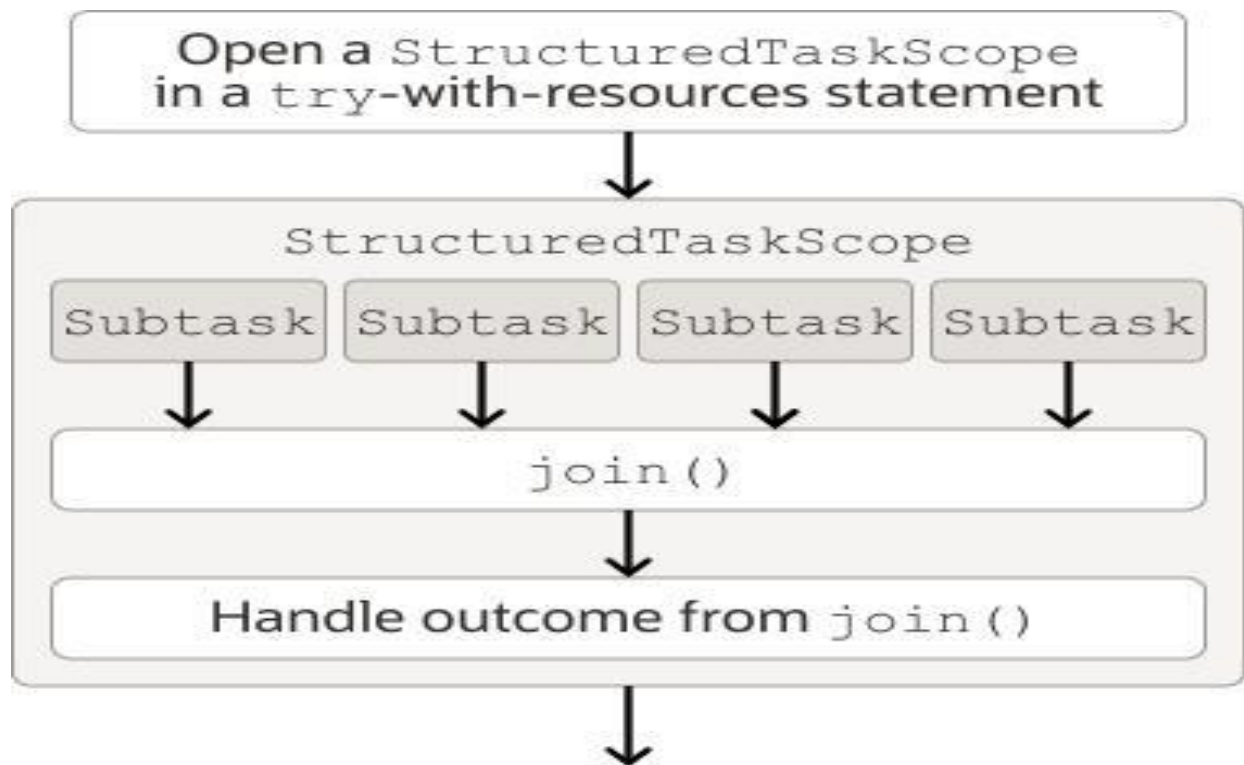


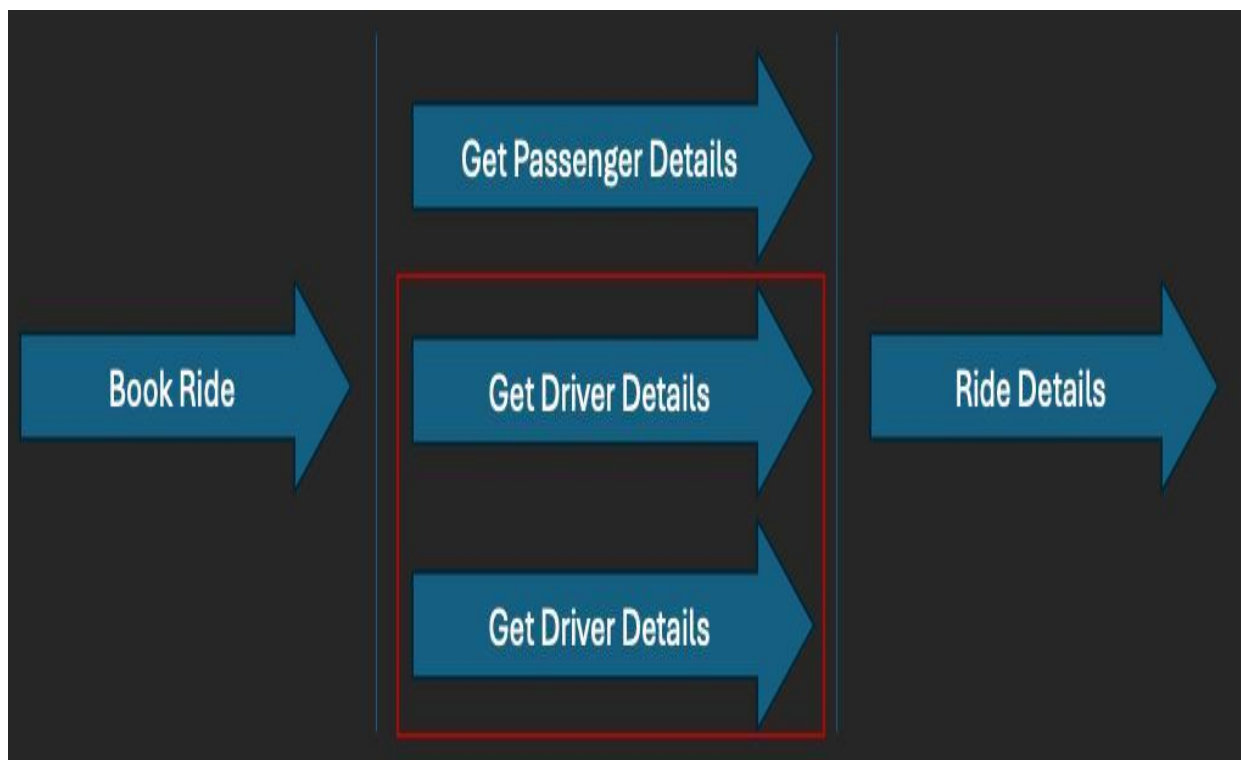
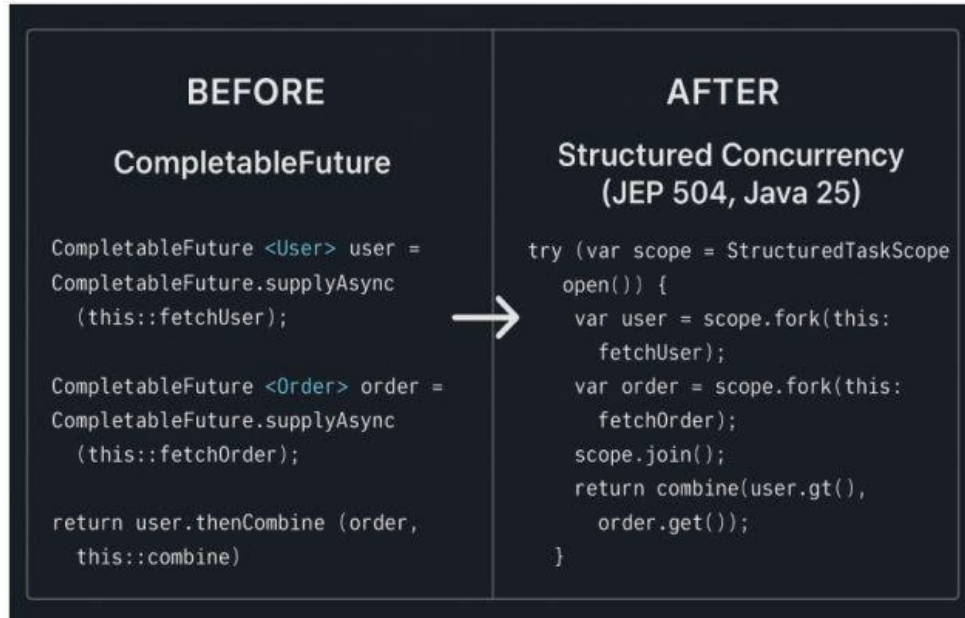


The parent doesn't leave the structured scope until its child tasks have been appropriately handled.

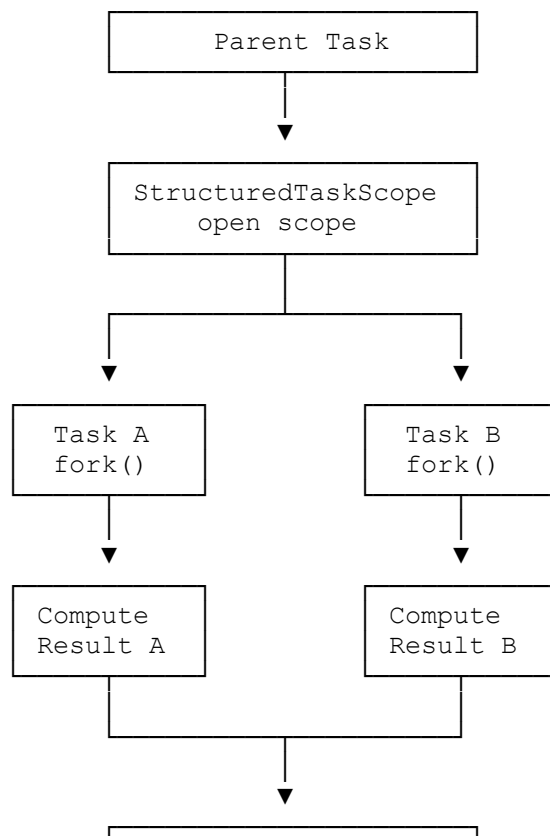
4. Flowchart

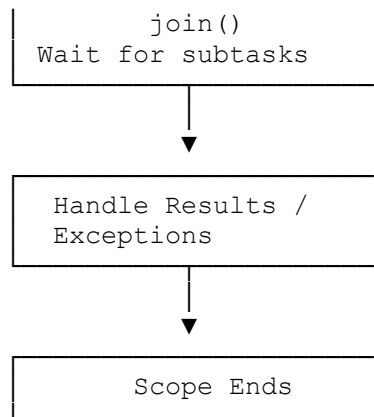






```
Response handle() throws InterruptedException {  
    try (var scope = StructuredTaskScope.open()) {  
        Subtask<String> user = scope.fork(() → findUser());  
        Subtask<Integer> order = scope.fork(() → fetchOrder());  
  
        scope.join();  
  
        return new Response(user.get(), order.get());  
    }  
}
```





5. Java 25 Code Example

Because this is a **preview API**, compile and run it with preview enabled.

```
import java.util.concurrent.StructuredTaskScope;

public class StructuredConcurrency {

    public static void main(String[] args)
        throws InterruptedException {

        try (var scope = StructuredTaskScope.open()) {

            var task1 = scope.fork(
                StructuredConcurrency::getUser
            );

            var task2 = scope.fork(
                StructuredConcurrency::getOrders
            );

            scope.join();

            String user = task1.get();
            String orders = task2.get();

            System.out.println("User    : " + user);
            System.out.println("Orders : " + orders);

        }

        private static String getUser() {

            sleep(1000);

            return "Anurag";

        }

    }

}
```

```

private static String getOrders() {
    sleep(1500);
    return "Order-1001, Order-1002";
}

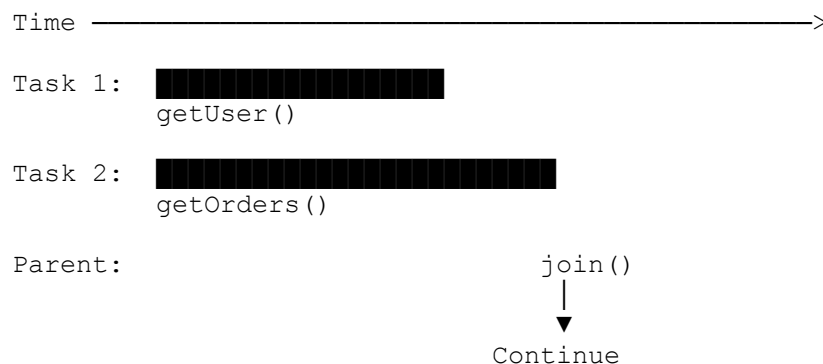
private static void sleep(long milliseconds) {
    try {
        Thread.sleep(milliseconds);
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
        throw new RuntimeException(e);
    }
}
}

```

Conceptual output

User : Anurag
Orders : Order-1001, Order-1002

The two operations can execute concurrently:



Instead of:

```

getUser()
↓
wait
↓
getOrders()
↓
wait
↓
continue

```

both operations can progress concurrently.

6. Failure Handling

One of the important benefits of structured concurrency is that failures can be handled at the scope level.

For example:

```
import java.util.concurrent.StructuredTaskScope;

public class StructuredConcurrency {

    public static void main(String[] args)
        throws InterruptedException {

        try (var scope = StructuredTaskScope.open(
            StructuredTaskScope.Joiner.allSuccessfulOrThrow())) {

            var userTask = scope.fork(
                StructuredConcurrency::getUser
            );

            var orderTask = scope.fork(
                StructuredConcurrency::getOrders
            );

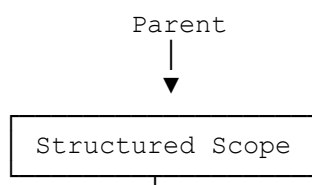
            scope.join();

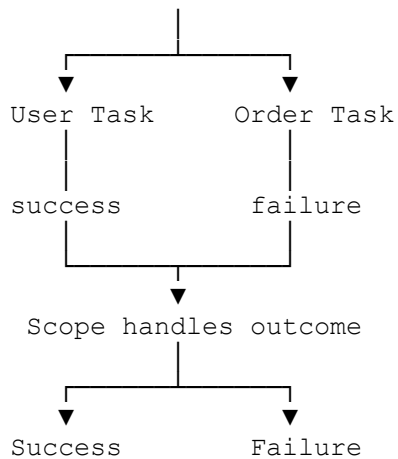
            System.out.println(userTask.get());
            System.out.println(orderTask.get());
        }

        static String getUser() {
            return "User-100";
        }

        static String getOrders() {
            return "Orders found";
        }
    }
}
```

The important conceptual relationship is:

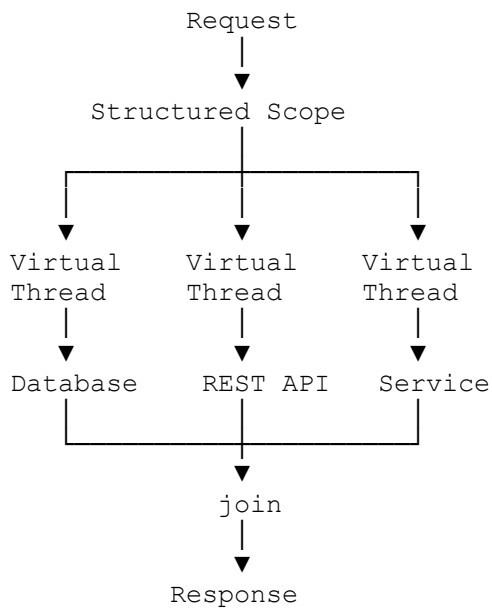




The scope provides a single place to reason about the group of related tasks.

7. Structured Concurrency + Virtual Threads

Structured Concurrency works particularly well with **virtual threads**.



This is useful in server-side applications where one request may need information from several independent services.

8. Key API Concepts

API concept	Purpose
StructuredTaskScope	Defines the structured concurrency scope
fork(...)	Starts a concurrent subtask
join()	Waits for subtasks
Joiner	Defines how task outcomes are combined/handled
close()	Closes the scope
try-with-resources	Ensures scope cleanup

A simplified mental model is:

```
try (var scope = StructuredTaskScope.open()) {  
  
    var taskA = scope.fork(...);  
    var taskB = scope.fork(...);  
  
    scope.join();  
  
    // process results  
}
```

9. Structured Concurrency vs Traditional Threads

Traditional concurrency	Structured concurrency
Tasks can become detached	Tasks have a defined parent scope
Manual lifecycle management	Scope manages task lifecycle
Cancellation can be complicated	Scope provides structured cancellation
Exceptions can be scattered	Scope-level failure handling
Harder to reason about task lifetime	Task lifetime follows lexical scope
Works with threads/executors	Particularly effective with virtual threads

10. Java 25 Preview Status

The evolution is important:

Java 19 → First Preview
Java 20 → Second Preview
Java 21 → Third Preview
Java 22 → Fourth Preview
Java 23 → Fifth Preview
Java 24 → Sixth Preview
Java 25 → Seventh Preview

Correction to the feature label in your question: Java 25's Structured Concurrency is **not the fifth preview**. JEP 505 is the **fifth preview** in the *JEP numbering/history sense*? Actually, the Java release history needs care here: Structured Concurrency was previewed in JDK 19, 20, 21, 22, and 23, then **sixth preview in JDK 24 and seventh preview in JDK 25**.

So for your Java-version catalog, the accurate entry is:

Java Version : 25
Year : 2025
Feature : Structured Concurrency (Seventh Preview)
UniqueClassName : StructuredConcurrency

Key takeaway

Structured Concurrency in Java 25 provides a structured way to run and coordinate related concurrent tasks, making task lifetime, cancellation, failure handling, and result management easier to reason about—especially when combined with virtual threads.

Java 25 — Scoped Values

Java Version: 25
Year: 2025
Feature: Scoped Values
UniqueClassName: ScopedValues

1. Definition

Scoped Values are a Java mechanism for sharing **immutable, dynamically scoped data** between methods and threads without explicitly passing that data through every method parameter.

In Java 25, Scoped Values are **finalized** through **JEP 506**. They were previously previewed in earlier Java releases.

A `ScopedValue<T>` represents a value that can be made available for a limited **dynamic scope** of execution.

The basic pattern is:

```
ScopedValue.where(KEY, value)
    .run() -> {
        // KEY is available here
    };
```

The value is:

- **Immutable** — it cannot be changed after binding.
 - **Scoped** — it is available only within a defined execution scope.
 - **Efficiently inherited** by child threads, including virtual threads.
 - Useful for request context, security context, transaction context, tracing IDs, and similar data.
-

2. The Problem Scoped Values Solve

Suppose an application has a request ID.

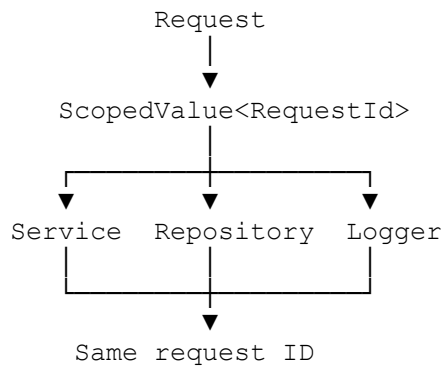
Without Scoped Values, you might have to pass it through many methods:

```
id="2b7q8d"
main(requestId)
  ↓
processRequest(requestId)
  ↓
service(requestId)
  ↓
repository(requestId)
  ↓
logger(requestId)
```

This is sometimes called **parameter plumbing**.

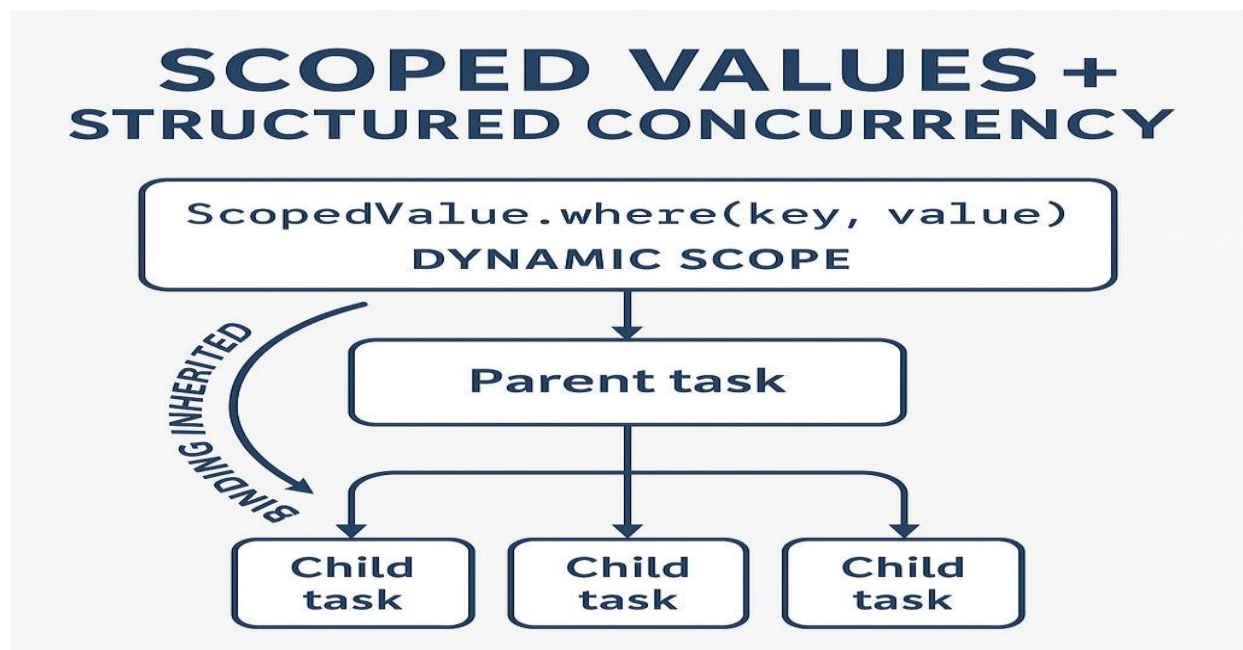
With Scoped Values:

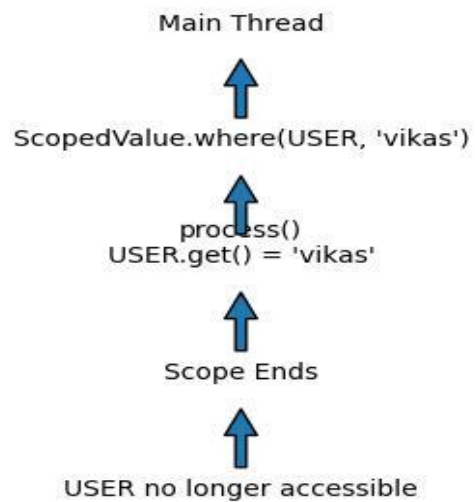
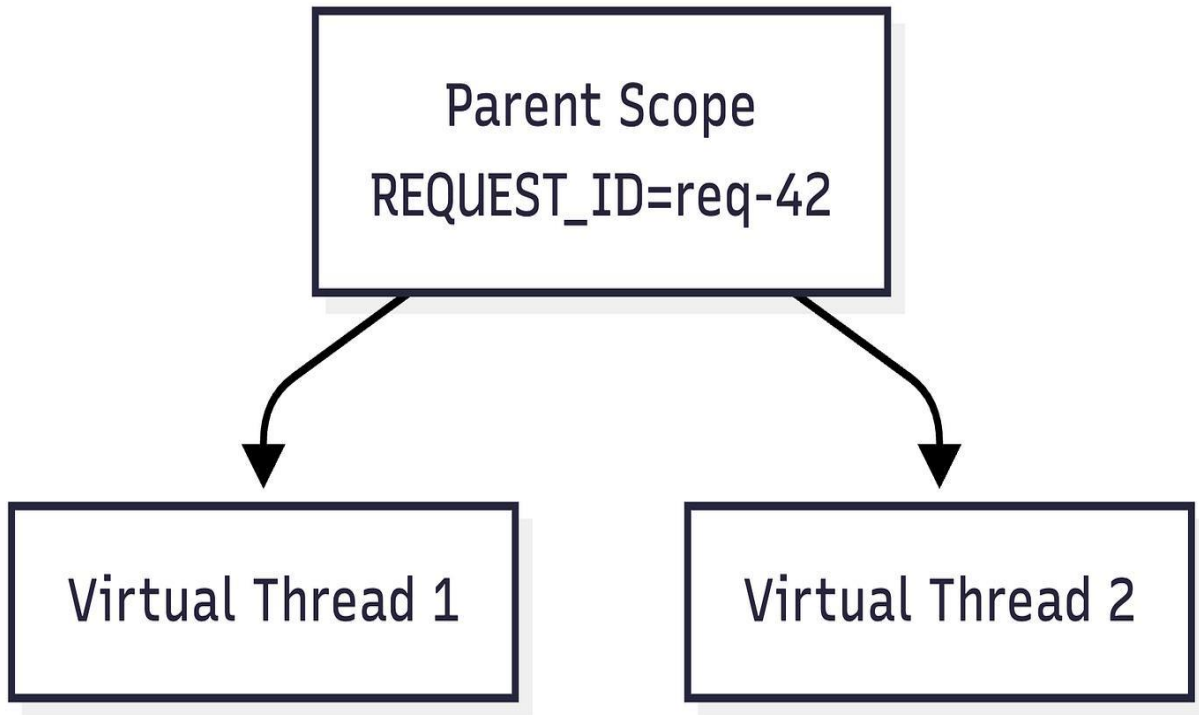
```
id="f9q1sc"
```

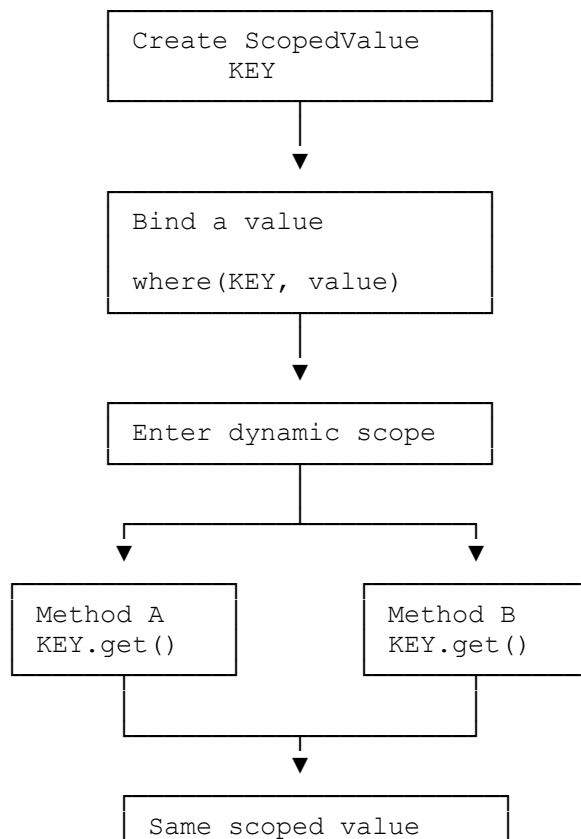
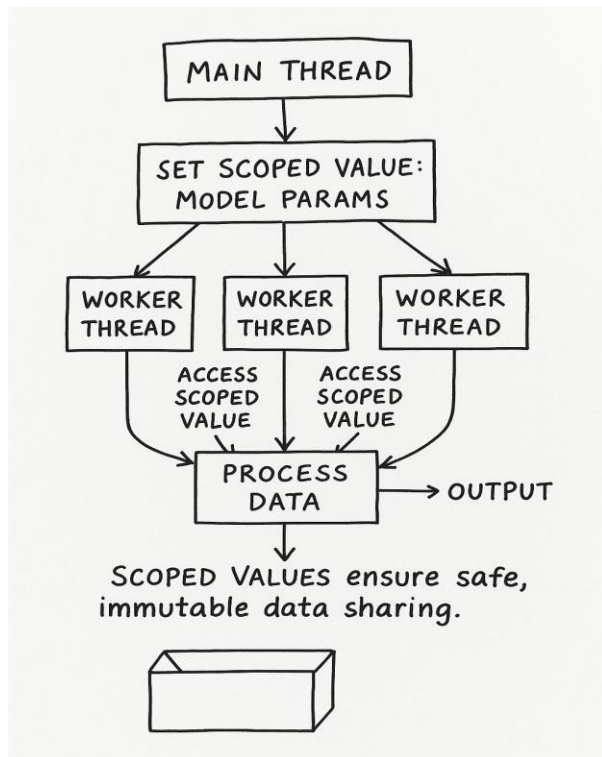


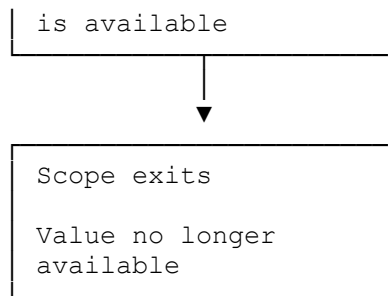
The methods can access the scoped value without receiving it as an explicit parameter.

3. Basic Flowchart









4. Java Code Example

Java 25 provides the `java.lang.ScopedValue` class.

```
public class ScopedValues {  
  
    private static final ScopedValue<String> USER =  
        ScopedValue.newInstance();  
  
    public static void main(String[] args) {  
  
        ScopedValue.where(USER, "Anurag")  
            .run() -> {  
  
            service();  
        });  
    }  
  
    static void service() {  
  
        System.out.println(  
            "Service user: " + USER.get()  
        );  
  
        repository();  
    }  
  
    static void repository() {  
  
        System.out.println(  
            "Repository user: " + USER.get()  
        );  
    }  
}
```

Output

Service user: Anurag

Repository user: Anurag

Notice that neither `service()` nor `repository()` receives the user as a parameter.

Instead:

`USER.get()`

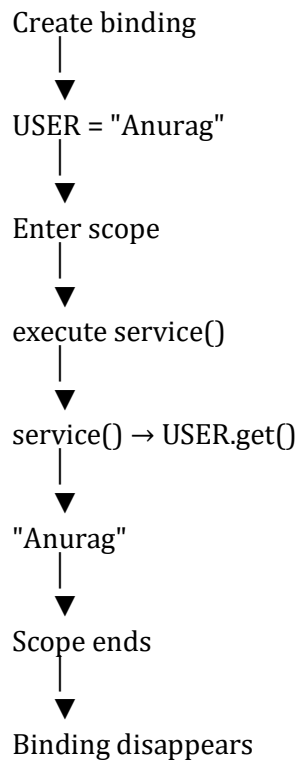
retrieves the value from the current scope.

5. How `where()` Works

This statement:

```
ScopedValue.where(USER, "Anurag")
    .run() -> {
        service();
    };
```

can be understood as:



6. Scoped Value Is Not a Global Variable

A ScopedValue may look somewhat like a global variable:

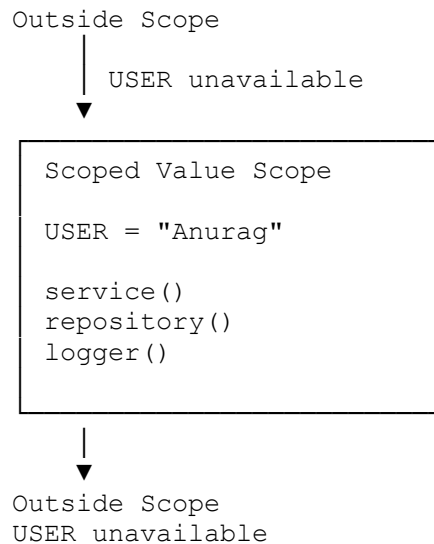
```
private static final ScopedValue<String> USER =  
    ScopedValue.newInstance();
```

But it is **not globally mutable state**.

The actual value exists only within a dynamic scope:

```
ScopedValue.where(USER, "Anurag")  
    .run(() -> {  
        // USER.get() works here  
    });  
  
// USER.get() here would fail because  
// the binding is no longer active.
```

This provides an important boundary:



7. call() Example

If the scoped operation needs to return a value, use `call()`:

```
public class ScopedValues {  
  
    private static final ScopedValue<String> USER =  
        ScopedValue.newInstance();  
  
    public static void main(String[] args)
```

```

throws Exception {

String result =
    ScopedValue.where(USER, "Anurag")
        .call() -> {

            return createMessage();
        });

    System.out.println(result);
}

static String createMessage() {

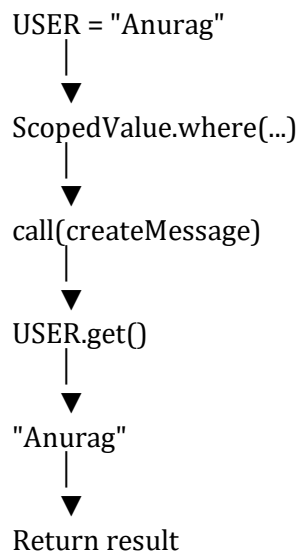
    return "Hello, " + USER.get();
}
}

```

Output

Hello, Anurag

The flow is:



8. Nested Scopes

Scoped Values can also be **re-bound in a nested scope**.

```
public class ScopedValues {  
  
    private static final ScopedValue<String> USER =  
        ScopedValue.newInstance();  
  
    public static void main(String[] args) {  
  
        ScopedValue.where(USER, "Anurag")  
            .run() -> {  
  
            System.out.println(USER.get());  
  
            ScopedValue.where(USER, "Admin")  
                .run() -> {  
  
                System.out.println(USER.get());  
            };  
  
            System.out.println(USER.get());  
        };  
    }  
}
```

Output

```
Anurag  
Admin  
Anurag
```

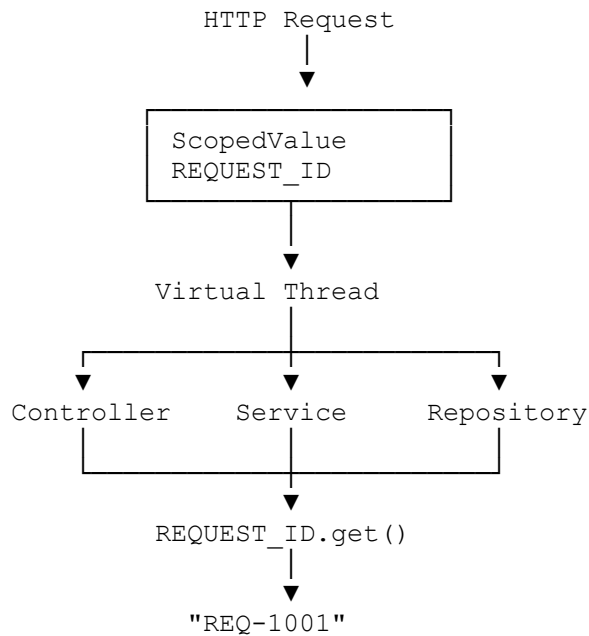
The flow is:

```
Outer Scope  
USER = "Anurag"  
    |  
    ▼  
Inner Scope  
USER = "Admin"  
    |  
    ▼  
Inner scope ends  
    |  
    ▼  
USER = "Anurag"
```

The inner binding temporarily **shadows** the outer binding.

9. Scoped Values with Virtual Threads

One of the important use cases is request processing with virtual threads.



This makes Scoped Values particularly useful for **server-side request context**.

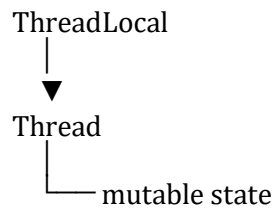
10. Scoped Values vs ThreadLocal

Scoped Values are often compared with ThreadLocal.

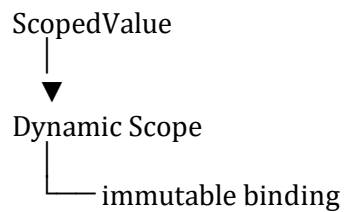
ThreadLocal	ScopedValue
Mutable per-thread variable	Immutable scoped binding
Value can be changed	Value cannot be changed
Lifetime associated with thread	Lifetime associated with dynamic scope
Often requires remove() discipline	Automatically leaves scope
Can be used for long-lived thread state	Designed for bounded, contextual data
Older mechanism	Modern Java mechanism

Useful for thread-local state	Useful for immutable context
-------------------------------	------------------------------

A conceptual difference is:



versus:



11. Common Use Cases

Scoped Values are particularly suitable for contextual information such as:

Request ID

```
ScopedValue<String> REQUEST_ID;
```

User identity

```
ScopedValue<String> USER;
```

Security context

```
ScopedValue<SecurityContext> SECURITY_CONTEXT;
```

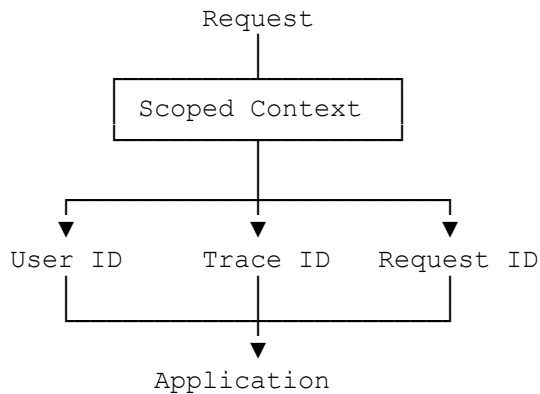
Transaction context

```
ScopedValue<TransactionContext> TRANSACTION;
```

Distributed tracing

```
ScopedValue<String> TRACE_ID;
```

Conceptually:



12. Important Java 25 Status

For your Java feature catalog, the important distinction is:

Java 20 → Scoped Values (Incubator)
Java 21 → Scoped Values (Second Preview)
Java 22 → Scoped Values (Second Preview)
Java 23 → Scoped Values (Third Preview)
Java 24 → Scoped Values (Fourth Preview)
Java 25 → Scoped Values (Final)

More precisely, **JEP 506 finalized Scoped Values in JDK 25** after its preview history.

Therefore your catalog entry can be recorded as:

Java Version	: 25
Year	: 2025
Feature	: Scoped Values
UniqueClassName	: ScopedValues
Status	: Final
JEP	: 506

Key Takeaway

Scoped Values in Java 25 provide an immutable, bounded mechanism for sharing contextual data across methods and threads without passing that data explicitly through every method parameter. They are especially useful with virtual threads and structured concurrency for request-scoped data such as user identity, tracing information, and security context.

Java 25 — Primitive Types in Patterns (Third Preview)

Java Version: 25

Year: 2025

Feature: Primitive Types in Patterns (Third Preview)

UniqueClassName: PrimitiveTypesPatterns

1. Definition

Primitive Types in Patterns is a **third preview feature in Java 25**, specified by **JEP 507**.

It enhances Java's **pattern matching** by allowing **primitive types to be used directly in type patterns, instanceof, and switch pattern matching**, including conversions between primitive types where appropriate.

Before this feature, Java's pattern matching was primarily designed around **reference types**.

For example:

```
if (obj instanceof Integer i) {  
    System.out.println(i);  
}
```

Java 25 expands the pattern-matching model so that primitive values can participate directly in patterns.

2. Why Primitive Patterns?

Consider a value:

```
int value = 42;
```

Traditional switch statements can test constants:

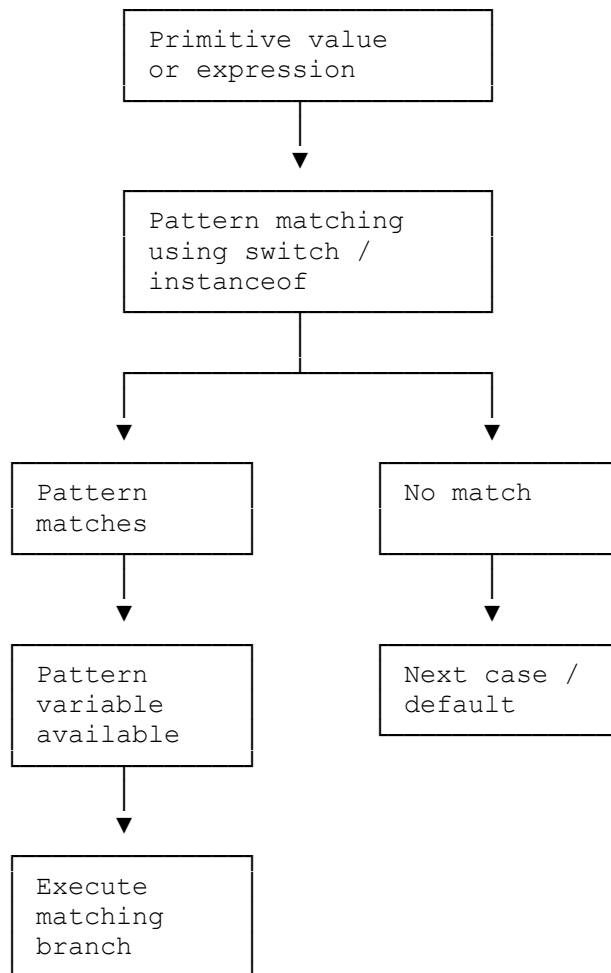
```
switch (value) {  
    case 0 -> System.out.println("Zero");  
    case 42 -> System.out.println("Answer");  
    default -> System.out.println("Other");  
}
```

Primitive type patterns allow the type itself to become part of the pattern:

```
switch (value) {  
    case int i -> System.out.println("Integer: " + i);  
}
```

This becomes particularly powerful when dealing with multiple primitive types and nested patterns.

3. Flowchart



4. Basic Java Example

Because this is a **preview feature**, compile and run the program with preview enabled.

```
public class PrimitiveTypesPatterns {  
  
    public static void main(String[] args) {  
  
        int value = 42;
```

```

switch (value) {
    case int i ->
        System.out.println(
            "Primitive int: " + i
        );
}
}
}

```

Output

Primitive int: 42

The important part is:

case int i ->

Here:

- int → primitive type
- i → pattern variable
- -> → matching case action

5. Primitive Type Patterns with switch

A more interesting example is:

```

public class PrimitiveTypesPatterns {

    static String classify(int value) {

        return switch (value) {

            case int i when i < 0 ->
                "Negative integer";

            case int i when i == 0 ->
                "Zero";

            case int i when i > 0 ->
                "Positive integer";

        };

    }

    public static void main(String[] args) {

        System.out.println(classify(-10));
    }
}

```

```
        System.out.println(classify(0));
        System.out.println(classify(25));
    }
}
```

Output

Negative integer
Zero
Positive integer

The pattern:

case int i

captures the primitive int value in i.

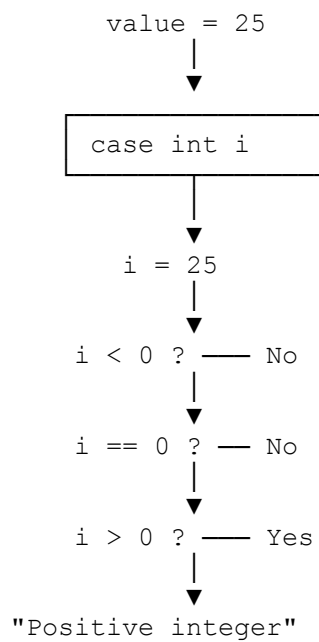
The when clause then applies an additional condition.

6. Flow of the Example

For:

classify(25)

the execution is conceptually:



7. Multiple Primitive Types

Primitive type patterns become particularly useful when a switch operates on a type that can represent different primitive values through the pattern-matching model.

For example, consider a boxed value:

```
public class PrimitiveTypesPatterns {  
    static String describe(Object value) {  
        return switch (value) {  
            case Integer i ->  
                "Integer: " + i;  
            case Double d ->  
                "Double: " + d;  
            case Long l ->  
                "Long: " + l;  
            default ->  
                "Other";  
        };  
    }  
  
    public static void main(String[] args) {  
        System.out.println(describe(10));  
        System.out.println(describe(10.5));  
        System.out.println(describe(100L));  
    }  
}
```

This example uses **reference wrapper types**, not the new primitive patterns themselves. It illustrates the older pattern-matching approach that primitive patterns extend.

8. Primitive Pattern and Numeric Conversion

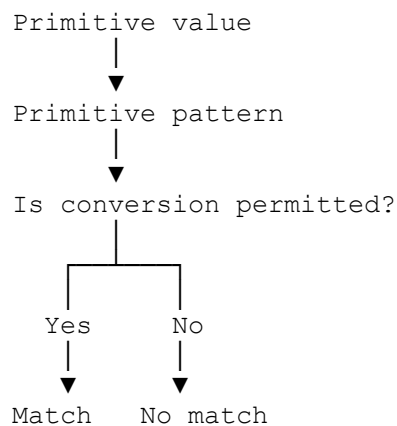
One of the important aspects of JEP 507 is that primitive patterns interact with Java's numeric conversion rules.

For example, primitive pattern matching can reason about values involving types such as:

byte
short
int
long
float
double
char
boolean

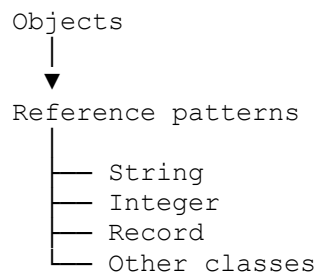
The feature addresses cases where primitive values need to be matched against patterns of different primitive types, while carefully defining when a conversion is safe.

Conceptually:

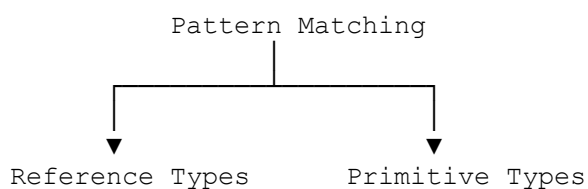


9. Why This Matters

Historically, Java pattern matching developed primarily around reference types:



Primitive patterns extend the model:





This makes Java's pattern-matching system more uniform.

10. Primitive Patterns with instanceof

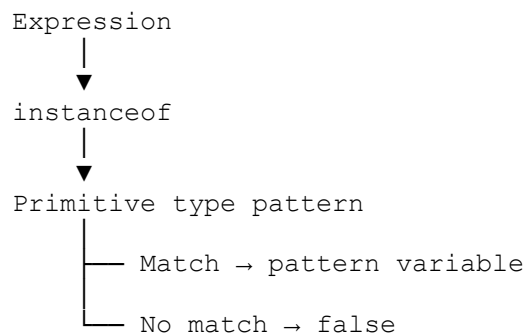
Primitive patterns also extend the possibilities around instanceof.

The traditional form is:

```
if (obj instanceof Integer i) {  
    System.out.println(i);  
}
```

With primitive patterns, Java can express primitive-oriented pattern matching more directly.

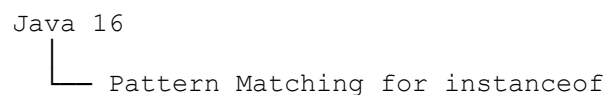
Conceptually:

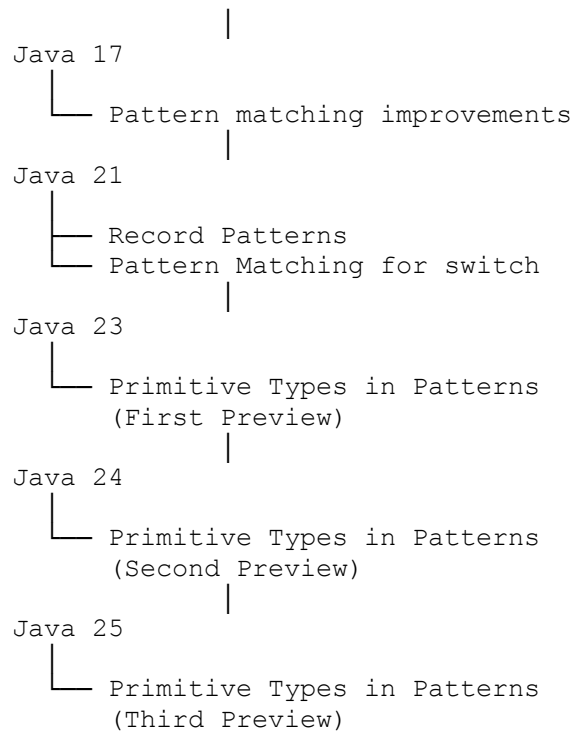


The exact legality of a particular instanceof expression depends on the operand and pattern types and Java's defined conversion rules.

11. Relationship with Other Pattern-Matching Features

Primitive Types in Patterns builds on Java's pattern-matching evolution:





12. Java 25 Catalog Entry

For your Java feature list:

Java Version: 25

Year: 2025

Feature: Primitive Types in Patterns (Third Preview)

UniqueClassName: PrimitiveTypesPatterns

JEP: 507

Status: Preview

Key takeaway

Primitive Types in Patterns (Third Preview) in Java 25 extends pattern matching so that primitive types can participate directly in pattern constructs, particularly switch, making Java's pattern-matching model more expressive and consistent across primitive and reference types.

Java 25 — Vector API (Tenth Incubator)

Java Version: 25

Year: 2025

Feature: Vector API (Tenth Incubator)

UniqueClassName: VectorAPI

JEP: 508

Status: Incubator

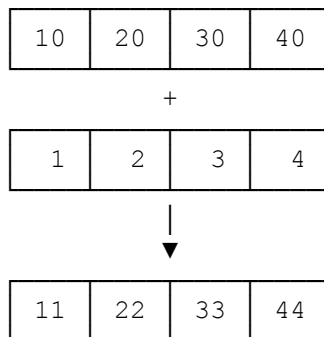
1. Definition

The **Vector API** is an incubating Java API for expressing **vector computations** that can be compiled by the JIT compiler into efficient CPU **SIMD (Single Instruction, Multiple Data)** instructions.

Instead of processing one value at a time:

```
a[0] + b[0]
a[1] + b[1]
a[2] + b[2]
a[3] + b[3]
```

vector operations can process multiple values together:



This can provide significant performance improvements for workloads involving large amounts of numerical data.

Typical applications include:

- scientific computing
 - machine learning
 - image/video processing
 - cryptography
 - financial calculations
 - signal processing
 - matrix/vector operations
-

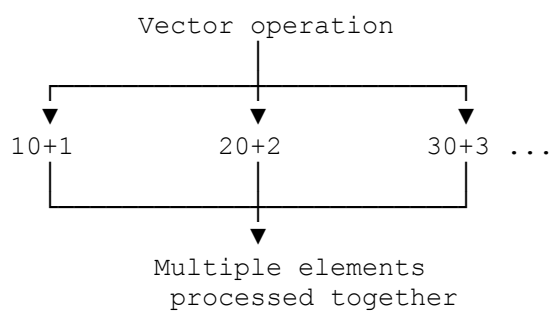
2. Scalar vs Vector Processing

Traditional scalar processing

Array A		Array B		Result
10		1		11
20		2		22
30	+	3	=	33
40		4		44

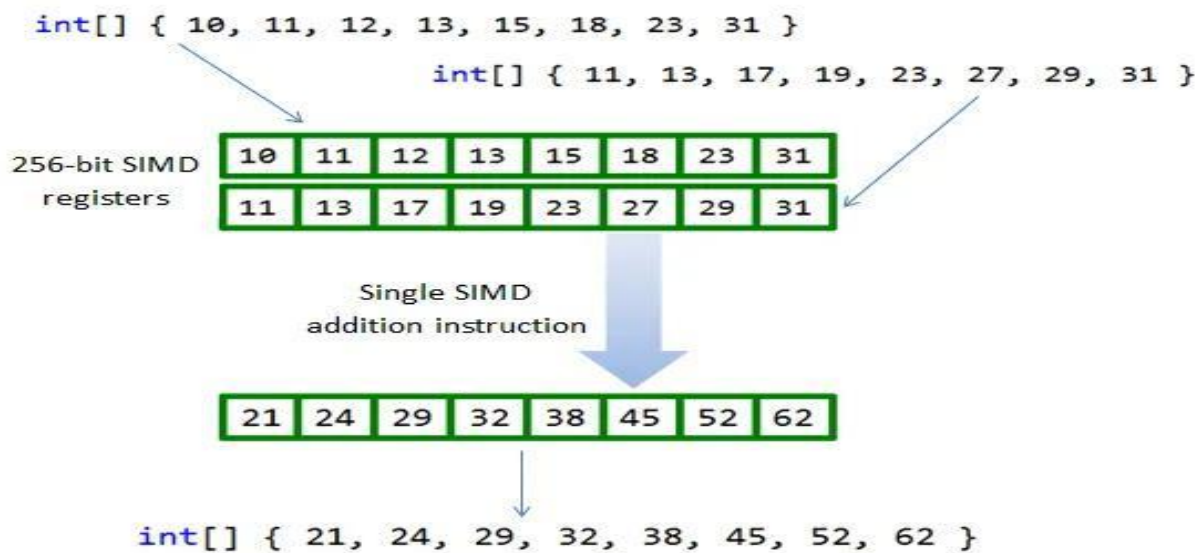
The CPU conceptually processes individual elements.

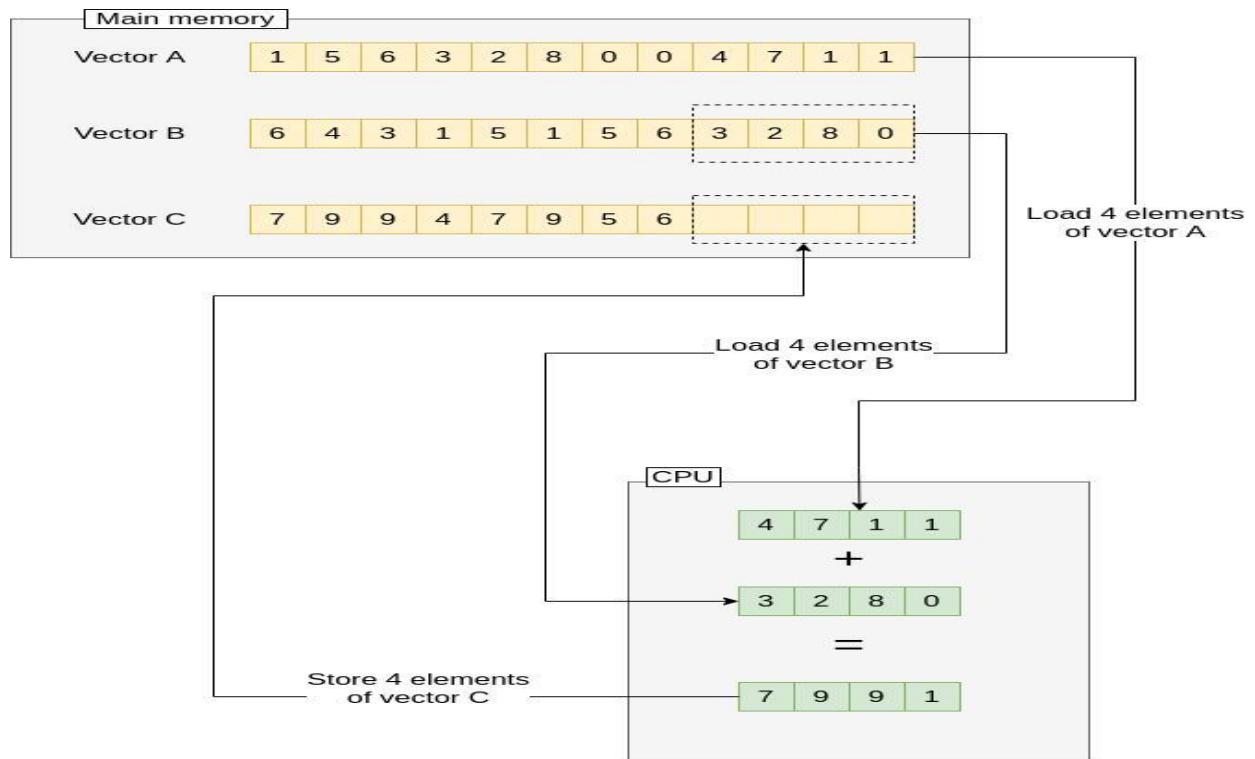
Vector processing



The exact degree of parallelism depends on the CPU's vector hardware and the selected vector species.

3. Flowchart





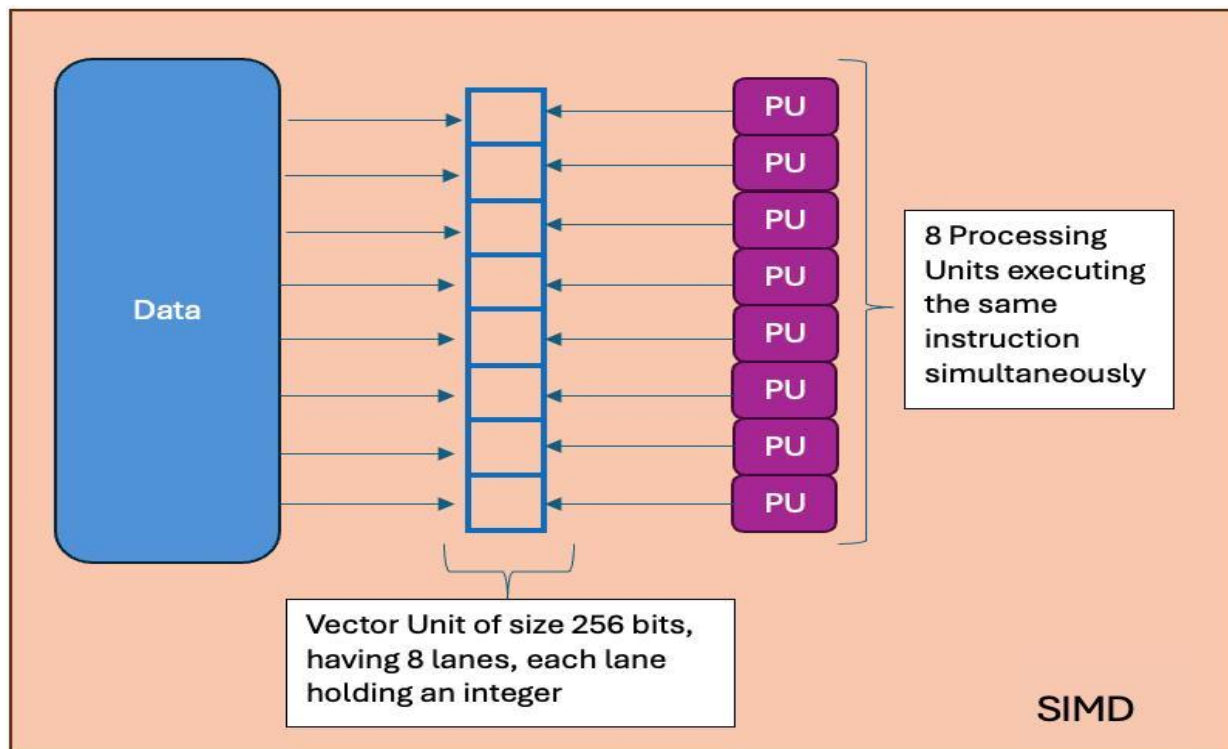
How is the Vector API Working?

Let us add two *vectors*

```
int[] v1 = ...; int[] v2 = ...;
int[] result = new int[v1.length];

var V1 = IntVector.fromArray(species, v1, 0);
var V2 = IntVector.fromArray(species, v2, 0);

var RESULT = V1.add(V2);
RESULT.intoArray(result, 0);
```

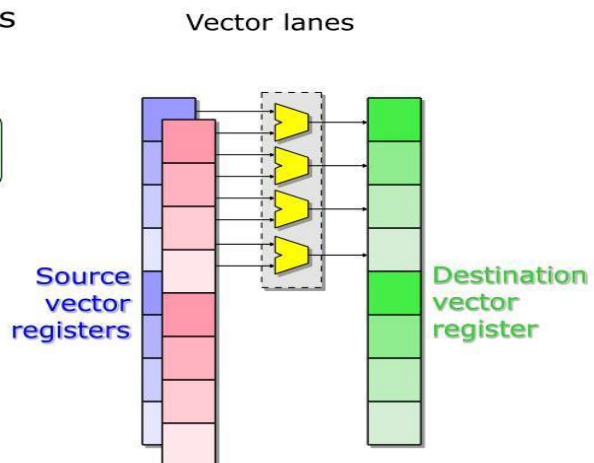


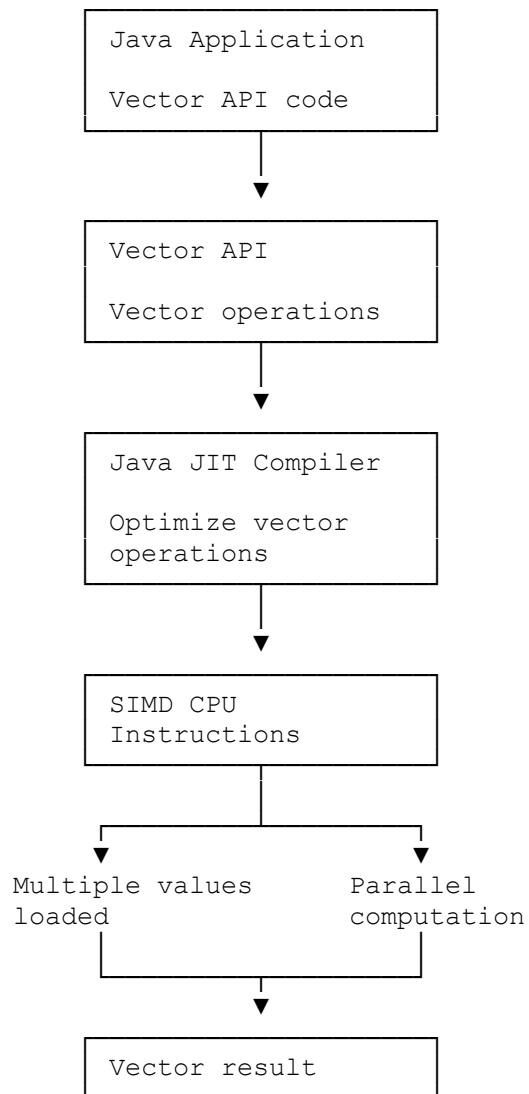
Acceleration with Vector Processing

- Organize data as long vectors
- **Data-level parallelism**

for (i=0; i<NELEM; i++) ➡ vmult a, b, c
 a[i] = b[i] * c[i]

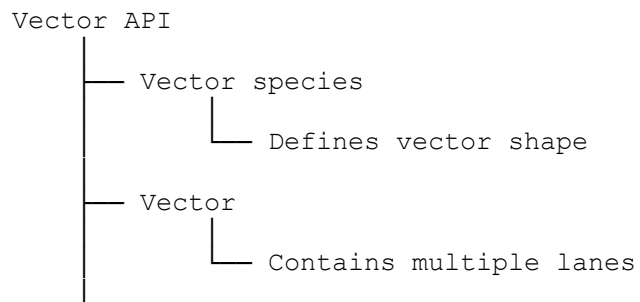
- Vector instruction execution
 - Multiple vector lanes (SIMD)
 - Repeated SIMD operation over length of vector

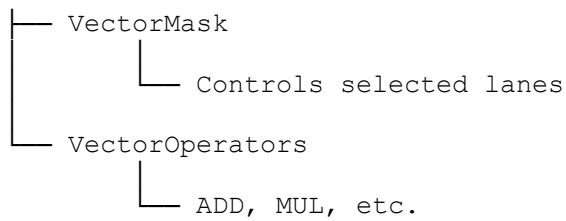




4. Main Vector API Concepts

The Vector API is centered around several important abstractions.





Vector

A vector contains multiple values called **lanes**.

For example:

IntVector

10	20	30	40
L0	L1	L2	L3

Vector Species

A VectorSpecies describes the vector shape and element type.

For example:

IntVector.SPECIES_PREFERRED

asks the JVM to use the preferred vector shape for the current platform.

5. Java Code Example — Vector Addition

Because Vector API is an **incubator module**, it is not part of the normal java.base module.

A simple example:

```
import jdk.incubator.vector.IntVector;

public class VectorAPI {

    public static void main(String[] args) {

        int[] a = {10, 20, 30, 40, 50, 60, 70, 80};
        int[] b = { 1,  2,  3,  4,  5,  6,  7,  8};
        int[] result = new int[a.length];

        var species = IntVector.SPECIES_PREFERRED;
```

```

    int i = 0;

    // Vectorized portion
    for (; i < species.loopBound(a.length);
        i += species.length()) {

        var va = IntVector.fromArray(
            species, a, i);

        var vb = IntVector.fromArray(
            species, b, i);

        var vr = va.add(vb);

        vr.intoArray(result, i);
    }

    // Remaining elements
    for (; i < a.length; i++) {
        result[i] = a[i] + b[i];
    }

    for (int value : result) {
        System.out.print(value + " ");
    }
}

```

Output

11 22 33 44 55 66 77 88

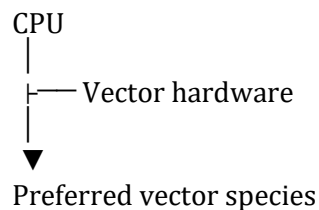
6. Understanding the Code

Step 1 — Select preferred species

```
var species = IntVector.SPECIES_PREFERRED;
```

The JVM selects an appropriate vector shape for the underlying hardware.

Conceptually:



↓
▼
IntVector

Step 2 — Load an array into a vector

```
var va = IntVector.fromArray(  
    species, a, i);
```

Instead of loading one integer:

`a[i]`

the vector operation loads several integers:

`a[i]    a[i+1]    a[i+2]    a[i+3] ...`

into vector lanes.

Step 3 — Add vectors

```
var vr = va.add(vb);
```

Conceptually:

```
va = [10, 20, 30, 40]
```

```
vb = [ 1,  2,  3,  4]
```

ADD

```
vr = [11, 22, 33, 44]
```

Step 4 — Store the result

```
vr.intoArray(result, i);
```

The vector is written back into the Java array:

Vector

11	22	33	44
----	----	----	----

↓
Java array

7. Vectorized Loop

This part is especially important:

```
for (; i < species.loopBound(a.length);  
    i += species.length()) {
```

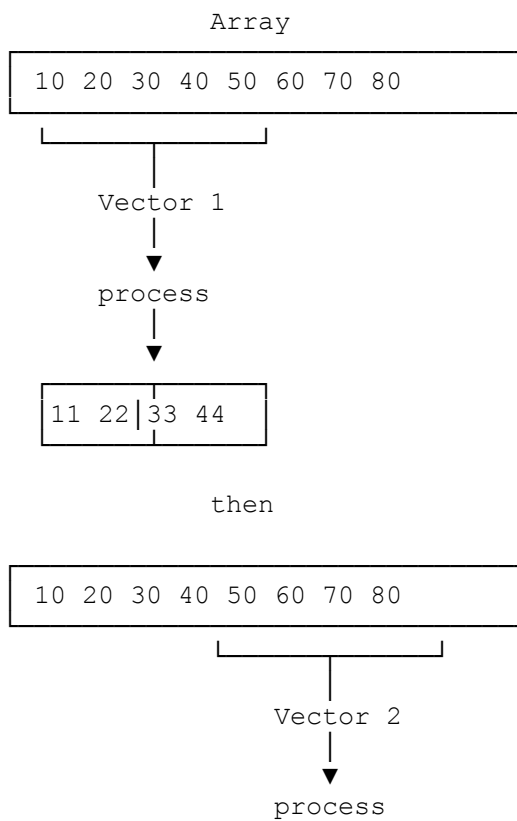
Suppose the vector species has four lanes:

Array length = 8

Iteration 1:
indexes → 0 1 2 3

Iteration 2:
indexes → 4 5 6 7

Flow:



The actual number of lanes is platform-dependent.

8. Vector API with Multiplication

The API isn't limited to addition.

```
import jdk.incubator.vector.IntVector;

public class VectorAPI {

    public static void main(String[] args) {

        int[] values = {1, 2, 3, 4, 5, 6, 7, 8};
        int[] result = new int[values.length];

        var species = IntVector.SPECIES_PREFERRED;

        int multiplier = 10;

        int i = 0;

        for (; i < species.loopBound(values.length);
            i += species.length()) {

            var vector =
                IntVector.fromArray(species, values, i);

            var resultVector =
                vector.mul(multiplier);

            resultVector.intoArray(result, i);
        }

        for (; i < values.length; i++) {
            result[i] = values[i] * multiplier;
        }

        for (int value : result) {
            System.out.print(value + " ");
        }
    }
}
```

Output:

10 20 30 40 50 60 70 80

9. Vector Mask

The API also supports conditional operations using VectorMask.

Conceptually:

Vector

10	20	30	40
----	----	----	----

Mask

true	false	true	false
------	-------	------	-------

The mask determines which lanes participate in an operation.

This is useful for vectorized conditional logic.

10. Scalar vs Vector Code

Scalar

```
for (int i = 0; i < a.length; i++) {  
    result[i] = a[i] + b[i];  
}
```

Conceptually:

10 + 1 → 11
20 + 2 → 22
30 + 3 → 33
40 + 4 → 44

Vector

```
var va = IntVector.fromArray(species, a, i);  
var vb = IntVector.fromArray(species, b, i);  
  
var resultVector = va.add(vb);
```

Conceptually:

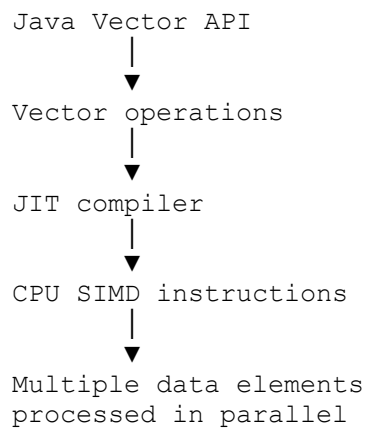
[10, 20, 30, 40]
+
[1, 2, 3, 4]

=
[11, 22, 33, 44]

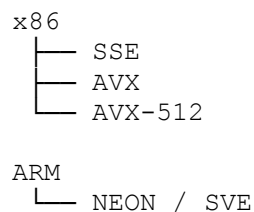
The vector form expresses the **parallel operation** while allowing the JIT compiler to map it to appropriate hardware instructions.

11. Vector API and SIMD

The overall relationship is:



Examples of CPU instruction families that may benefit include:



The application normally doesn't need to write architecture-specific assembly.

12. Why Use Vector API?

Without Vector API

You may have to rely on:

ordinary Java loops
|



JIT auto-vectorization

The JIT may or may not be able to recognize and vectorize a particular computation.

With Vector API

You explicitly express:

This operation is naturally vectorized.

Then:

Java Vector API



JIT optimization



Best available SIMD instructions

This gives developers a more explicit and portable way to express SIMD algorithms.

13. Important Java 25 Status

For your Java feature catalog:

Java Version	: 25
Year	: 2025
Feature	: Vector API (Tenth Incubator)
UniqueClassName	: VectorAPI
JEP	: 508
Status	: Incubator

The Vector API has been incubating across multiple JDK releases:

- Java 16 → First Incubator
- Java 17 → Second Incubator
- Java 18 → Third Incubator
- Java 19 → Fourth Incubator
- Java 20 → Fifth Incubator
- Java 21 → Sixth Incubator
- Java 22 → Seventh Incubator
- Java 23 → Eighth Incubator
- Java 24 → Ninth Incubator
- Java 25 → Tenth Incubator

Key Takeaway

Vector API (Tenth Incubator) in Java 25 provides a portable Java API for expressing SIMD-style computations over multiple data elements at once. The JIT compiler can translate these operations into efficient hardware vector instructions, making the API valuable for computationally intensive workloads such as numerical processing, machine learning, image processing, and cryptography.

Java 25 — JFR CPU-Time Profiling

Java Version: 25

Year: 2025

Feature: JFR CPU-Time Profiling

UniqueClassName: JFRCpuTimeProfiling

JEP: 509

Status: Experimental

JEP 509 introduces **CPU-time-based profiling to Java Flight Recorder (JFR)**. Unlike traditional JFR execution-time sampling, which samples threads based on elapsed/wall-clock time, CPU-time profiling samples based on the **actual CPU time consumed by a thread**. This can make CPU-bound behavior easier to identify, particularly when applications have many threads that spend significant time blocked or sleeping. (openjdk.org)

1. Definition

JFR CPU-Time Profiling is a Java 25 JFR capability that provides a CPU-time-based sampling profiler.

The basic idea is:

Traditional sampling

Wall-clock time



Sample thread

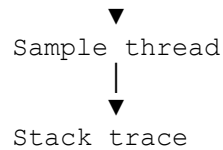


Stack trace

CPU-time profiling

CPU actually consumed





This distinction is important.

Suppose an application has:

Thread A → actively executing on CPU

Thread B → waiting for database

Thread C → sleeping

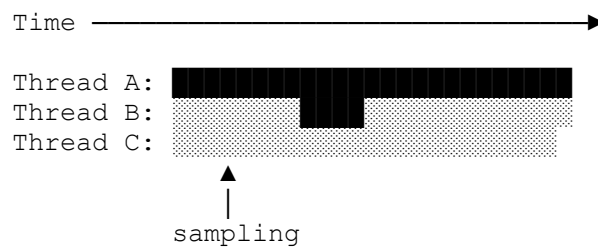
Thread D → waiting for I/O

A wall-clock sampler can encounter all of these threads.

CPU-time profiling focuses sampling effort on **threads actually consuming CPU**, making the resulting profile more representative of CPU usage.

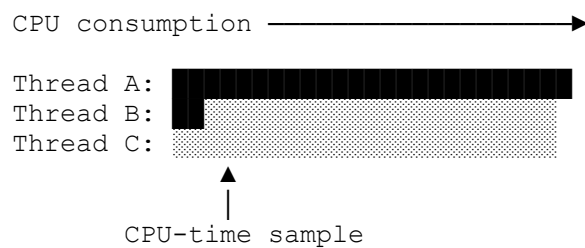
2. CPU Time vs Wall-Clock Time

Wall-clock profiling



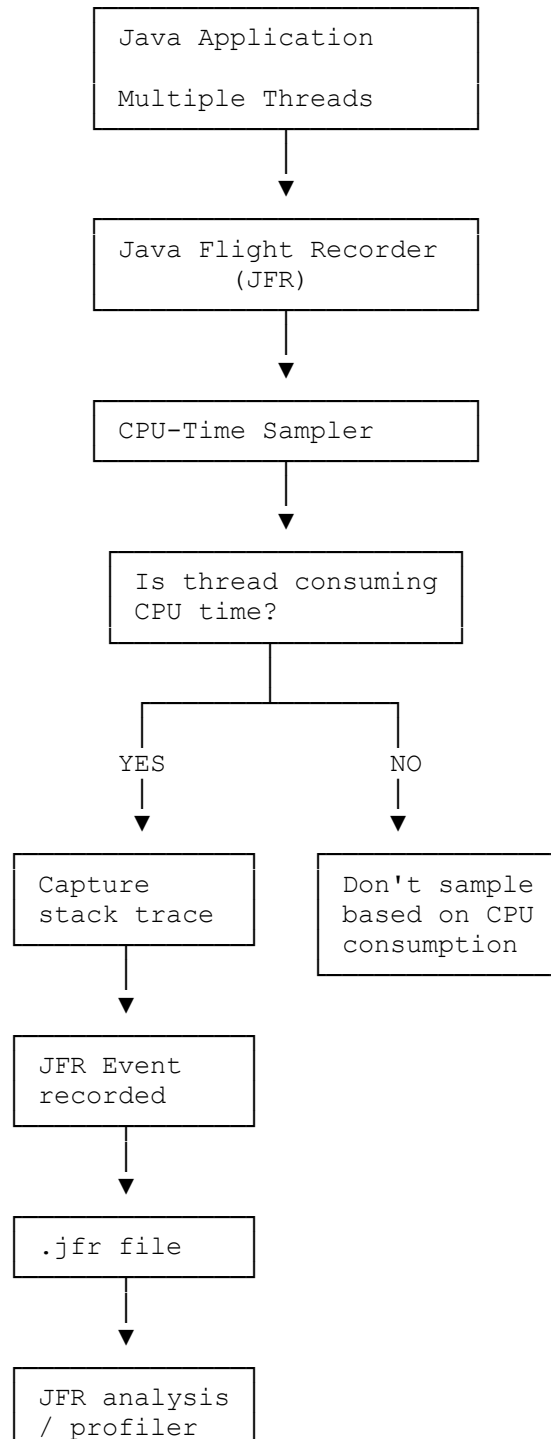
The profiler samples according to elapsed time.

CPU-time profiling



The sampling is driven by **CPU time consumed by the thread**, rather than simply the passage of wall-clock time.

3. Flowchart



The implementation uses CPU-time sampling infrastructure inside HotSpot/JFR; OpenJDK's implementation work describes a dedicated CPU-time sampler and CPU-time-based JFR sample requests. (mail.openjdk.org)

4. Java Code Example

Unlike many Java language features, **JFR CPU-Time Profiling does not introduce a normal application-level Java class that you call directly.**

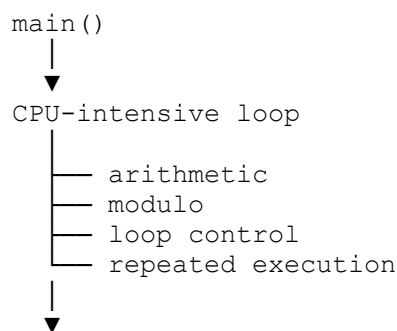
You normally enable it through **JFR configuration/command-line options** and then analyze the resulting .jfr recording.

For example, first create a CPU-intensive application:

```
public class JFRCpuTimeProfiling {  
  
    public static void main(String[] args) {  
  
        System.out.println("Starting CPU-intensive work...");  
  
        long result = 0;  
  
        for (long i = 0; i < 5_000_000_000L; i++) {  
            result += i % 97;  
        }  
  
        System.out.println("Result = " + result);  
    }  
}
```

The application deliberately performs CPU-intensive work.

Conceptually:



5. Starting a JFR Recording

You can start a JFR recording using the java command.

For example:

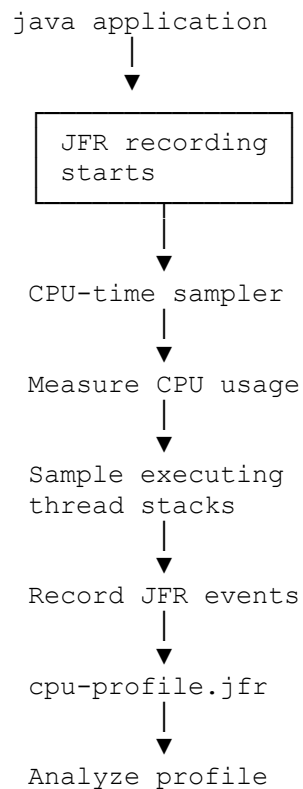
```
java -XX:StartFlightRecording=duration=30s,filename=cpu-profile.jfr JFRCpuTimeProfiling
```

This starts JFR for 30 seconds and writes the recording to:

cpu-profile.jfr

The Java launcher supports parameters such as duration and filename for Flight Recorder recordings. ([OpenJDK](#))

6. Conceptual CPU-Time Profiling Flow

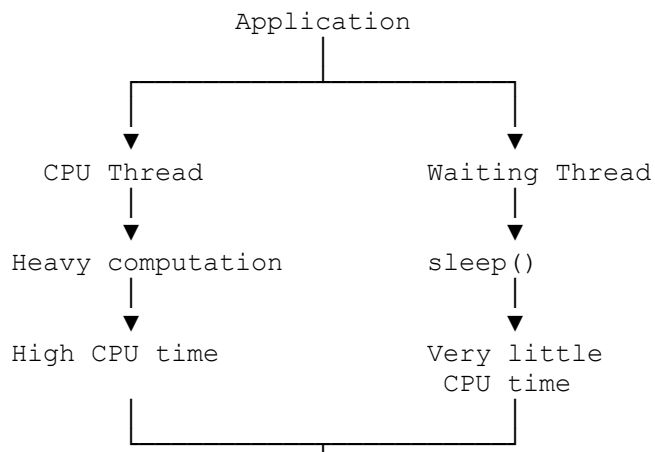


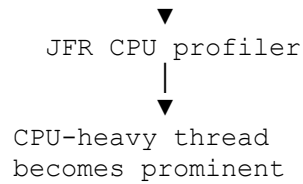
7. Example with Multiple Threads

CPU-time profiling becomes particularly useful when different threads have very different CPU behavior.

```
public class JFRCpuTimeProfiling {  
  
    public static void main(String[] args)  
        throws InterruptedException {  
  
        Thread cpuThread = new Thread(() -> {  
  
            long result = 0;  
  
            for (long i = 0; i < 5_000_000_000L; i++) {  
                result += i % 97;  
            }  
  
            System.out.println("CPU result: " + result);  
        });  
  
        Thread waitingThread = new Thread(() -> {  
  
            try {  
                Thread.sleep(20_000);  
            } catch (InterruptedException e) {  
                Thread.currentThread().interrupt();  
            }  
        });  
  
        cpuThread.start();  
        waitingThread.start();  
  
        cpuThread.join();  
        waitingThread.join();  
    }  
}
```

The conceptual behavior is:





This is one of the situations where CPU-time-based profiling can provide a useful view of where the processor is actually being consumed.

8. Traditional JFR Sampling vs CPU-Time Sampling

Aspect	Traditional execution-time sampling	CPU-time profiling
Sampling basis	Elapsed/wall-clock time	CPU time consumed
Sleeping threads	Can be encountered	CPU-time sampling de-emphasizes them
I/O-waiting threads	Can appear in samples	Less relevant to CPU consumption
CPU-bound code	Appears	Directly targeted
Main purpose	General execution profiling	CPU consumption profiling
JFR integration	Existing	JEP 509

9. Why It Is Useful

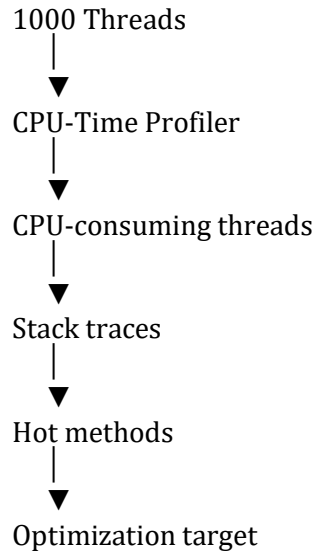
Imagine a server with 1,000 threads:

1000 threads
|
├— 50 CPU-intensive
├— 300 waiting for I/O
├— 400 waiting for locks
├— 200 sleeping
└— 50 miscellaneous

If your question is:

"Which code is actually consuming CPU?"

CPU-time profiling provides a sampling model specifically suited to that question.



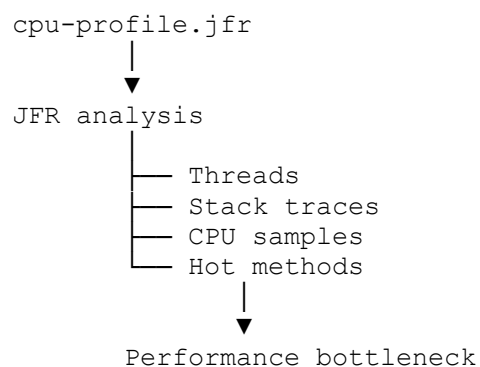
10. JFR Recording Analysis

After recording:

`cpu-profile.jfr`

you can inspect it using JDK Flight Recorder tooling, including the `jfr` command-line tool or JDK Mission Control.

Conceptually:



11. Important Java 25 Status

For your Java feature catalog, the accurate entry is:

Java Version : 25
Year : 2025
Feature : JFR CPU-Time Profiling (Experimental)
UniqueClassName : JFRCpuTimeProfiling
JEP : 509
Status : Experimental

OpenJDK's JEP index lists **JEP 509: JFR CPU-Time Profiling (Experimental)** as a Java 25 feature. (openjdk.org)

There was also implementation work and subsequent fixes during JDK 25 development, so this feature should not be treated as a conventional finalized Java API. (bugs.openjdk.org)

Key Takeaway

JFR CPU-Time Profiling in Java 25 adds CPU-time-based sampling to Java Flight Recorder, allowing developers to profile where threads actually spend CPU time rather than sampling purely according to elapsed time. It is especially useful for identifying CPU-intensive methods in applications with many blocked, sleeping, or I/O-bound threads.

Java 25 — Key Derivation Function API

Java Version: 25
Year: 2025
Feature: Key Derivation Function (KDF) API
UniqueClassName: KeyDerivationFunctionAPI
JEP: 510
Status: Final

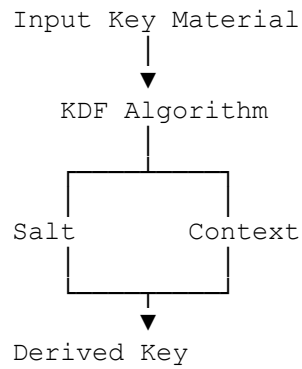
1. Definition

The **Key Derivation Function API** is a standardized Java API for deriving one or more cryptographic keys from an existing secret, such as:

- a password,
- a shared secret,
- a secret key,
- or other keying material.

It was introduced in **Java 24 as a preview** and became **final in Java 25** through **JEP 510**.

A **Key Derivation Function (KDF)** transforms input key material into cryptographically useful derived key material:

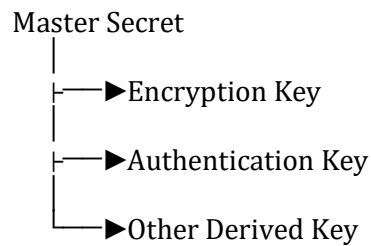


The API provides a standard JDK abstraction instead of requiring applications to implement KDF logic themselves or depend on third-party cryptographic libraries.

2. Why Do We Need a KDF?

A secret should generally not be reused directly for every cryptographic purpose.

For example:



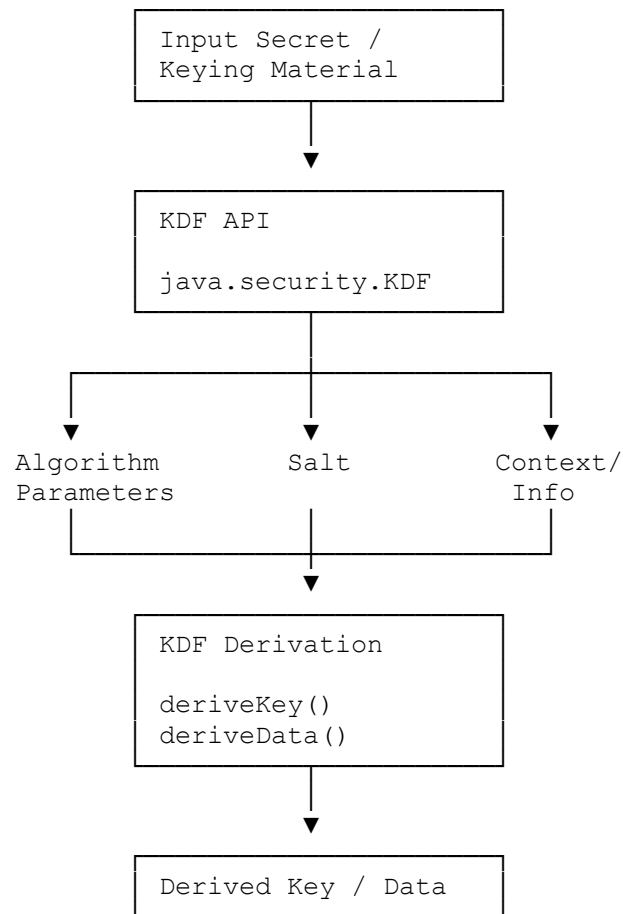
A KDF allows these keys to be derived in a controlled and cryptographically appropriate way.

A KDF is **not encryption**.

Encryption:
Plaintext + Key
↓
Ciphertext

KDF:
Secret + Parameters
↓
Derived Key

3. Flowchart



4. Important Java API Classes

The main API is in:

java.security

The central class is:

KDF

A simplified API relationship is:

```
java.security.KDF
|
├── KDF.getInstance(...)
|
└── deriveKey(...)
```

└─ deriveData(...)

Supporting classes include:

KDF
KDFParameters
KDFParameterSpec

The exact parameter specification depends on the KDF algorithm being used.

5. Java Code Example — HKDF

One important KDF is **HKDF (HMAC-based Extract-and-Expand Key Derivation Function)**.

Java 25 provides a standard KDF API for algorithms such as HKDF when supported by the provider.

A conceptual Java example is:

```
import java.nio.charset.StandardCharsets;
import java.security.KDF;
import javax.crypto.SecretKey;
import javax.crypto.spec.SecretKeySpec;

public class KeyDerivationFunctionAPI {

    public static void main(String[] args) throws Exception {

        // Input key material
        byte[] secret =
            "my-secret-key-material"
                .getBytes(StandardCharsets.UTF_8);

        // Create KDF
        KDF kdf = KDF.getInstance("HKDF-SHA256");

        // Example: derive 256-bit key
        byte[] derived =
            kdf.deriveData(
                secret,
                32
            );

        SecretKey key =
            new SecretKeySpec(
                derived,
                "AES"
            );

        System.out.println(
            "Derived key algorithm: "
        )
```

```

        + key.getAlgorithm()
    );

    System.out.println(
        "Derived key length: "
        + (key.getEncoded().length * 8)
        + " bits"
    );
}
}

```

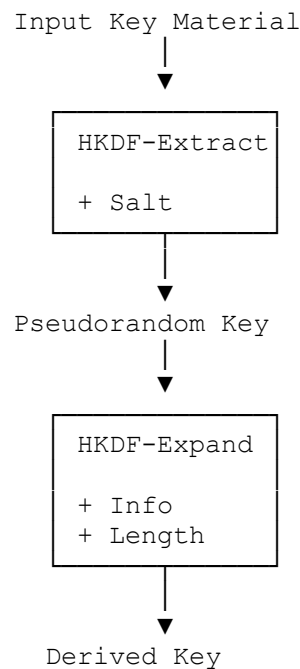
Conceptual output

Derived key algorithm: AES
Derived key length: 256 bits

Important: The exact KDF algorithm name and parameter API are provider-dependent; applications should query the installed provider's supported KDF algorithms rather than assume every provider exposes the same names.

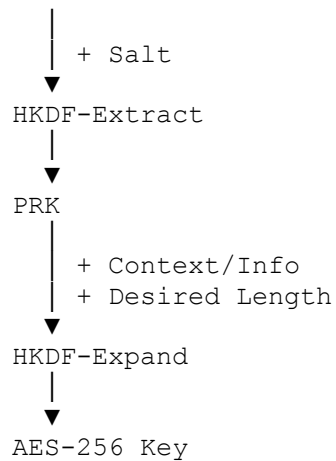
6. More Realistic HKDF Flow

HKDF conceptually has two stages:



For example:

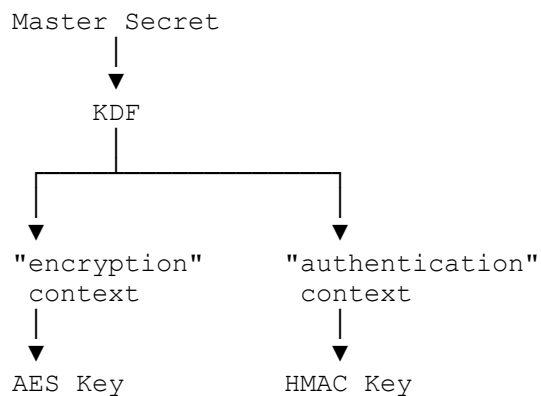
Secret



7. Deriving Different Keys from One Secret

One powerful use of a KDF is **key separation**.

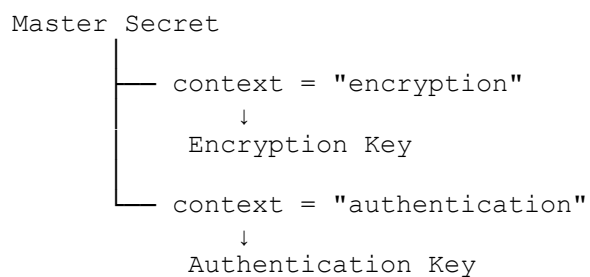
Suppose an application has:



The same underlying secret can therefore produce different keys for different purposes.

The **context/info value** distinguishes the purposes.

For example:



This is preferable to simply copying the same key for unrelated cryptographic purposes.

8. KDF vs Hash vs Encryption

These concepts are related but different.

Operation	Purpose
Hash	Produces a fixed-size digest
Encryption	Protects plaintext using a key
KDF	Derives cryptographic key material
MAC	Provides authentication/integrity
Password KDF	Derives keys from passwords while resisting attacks

Conceptually:

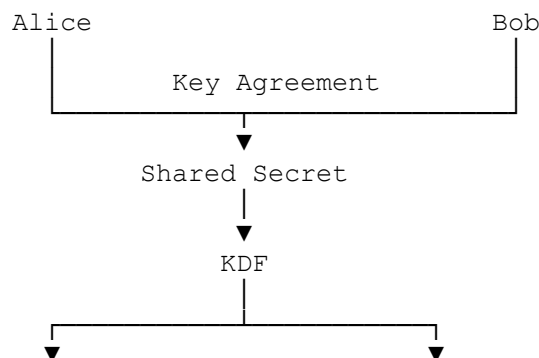
Hash:
Input $\longrightarrow$ Digest

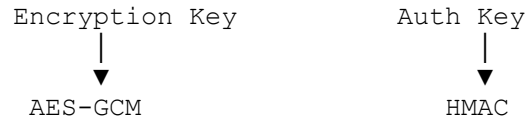
Encryption:
Plaintext + Key $\rightarrow$ Ciphertext

KDF:
Secret + Parameters $\rightarrow$ Derived Key

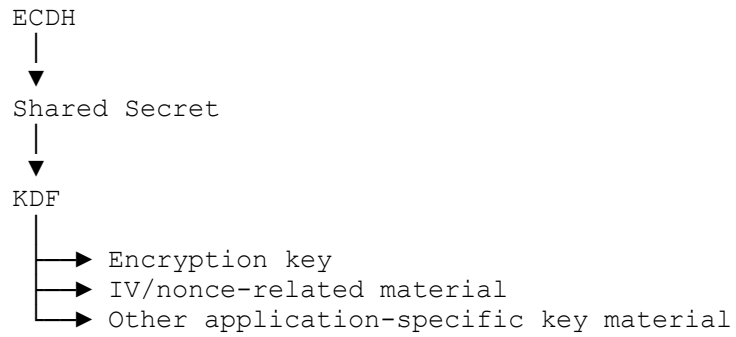
9. KDF with a Shared Secret

A common modern cryptographic workflow is:





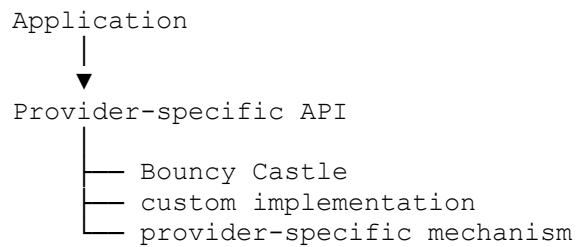
For example, after an ECDH key agreement:



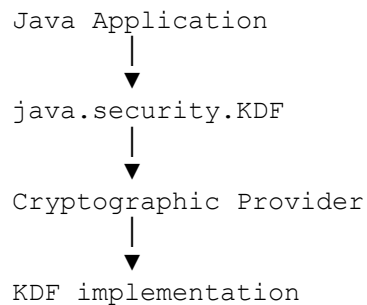
This is one of the important practical applications of the KDF API.

10. Why the Java KDF API Is Important

Before the standardized API, Java developers often had to:



With the standard API:



This gives applications a standard JDK abstraction while allowing the cryptographic provider to supply the actual implementation.

11. Security Considerations

A KDF does **not automatically make an insecure secret secure**.

For example:

"password123"

should not simply be treated as high-quality cryptographic key material.

For passwords, use a password-oriented KDF designed for password hashing/key derivation, with appropriate parameters and a unique salt.

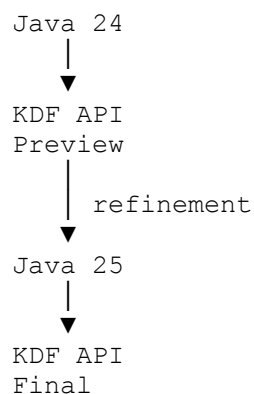
For high-entropy secrets, KDFs such as HKDF are commonly used for deriving application-specific keys.

Also:

- use an appropriate salt where the algorithm requires/recommends it,
 - use distinct context/info values for different purposes,
 - derive the required key length rather than arbitrary oversized material,
 - use well-reviewed cryptographic algorithms,
 - avoid implementing cryptographic primitives yourself.
-

12. Java 24 → Java 25

The evolution of this feature is:



For your Java feature catalog:

Java Version	: 25
Year	: 2025
Feature	: Key Derivation Function API
UniqueClassName	: KeyDerivationFunctionAPI
JEP	: 510
Status	: Final

Key Takeaway

The Key Derivation Function API in Java 25 provides a standard `java.security.KDF` API for deriving cryptographic keys and keying material from secrets. It became final in Java 25 through JEP 510 and is particularly useful for key separation, HKDF-based derivation, and deriving keys from shared secrets produced by key-agreement protocols.